МІНІСТЕРСТВО ОСВІТИ ТА НАУКИ УКРАЇНИ
НАЦІОНАЛЬНИЙ ТЕХНІЧНИЙ УНІВЕРСИТЕТ УКРАЇНИ
«КИЇВСЬКИЙ ПОЛІТЕХНІЧНИЙ ІНСТИТУТ ІМЕНІ ІГОРЯ СІКОРСЬКОГО»
НАВЧАЛЬНО-НАУКОВИЙ ІНСТИТУТ АТОМНОЇ ТА ТЕПЛОВОЇ ЕНЕРГЕТИКИ
КАФЕДРА ІНЖЕНЕРІЇ ПРОГРАМНОГО ЗАБЕЗПЕЧЕННЯ В ЕНЕРГЕТИЦІ

ЗВІТ З ПРАКТИЧНОЇ РОБОТИ №2: МЕНЮ, ФОРМИ, MVVM, ЛОКАЛЬНЕ СХОВИЩЕ ДАНИХ

з дисципліни: «РОЗРОБКА ПРОГРАМНОГО ЗАБЕЗПЕЧЕННЯ МОБІЛЬНИХ ПРИСТРОЇВ» на тему:

«Атомна електростанція з радіаційним моніторингом»

Виконав студент 4 курсу групи ТВ-33
Домарацький Дмитро Олександрович

Посилання на GitHub репозиторій

dmtrDO/mobile

Київ — 2026

ПРАКТИЧНЕ ЗАНЯТТЯ № 5

Тема: Створення головного меню мобільного застосунку та реалізація навігації між екранами за допомогою Jetpack Navigation Component у Compose.

Мета: Ознайомити з принципами організації навігації в Android застосунках, навчитися створювати головне меню та переходи між екранами.

Просунутий рівень (рисунок 1.1)

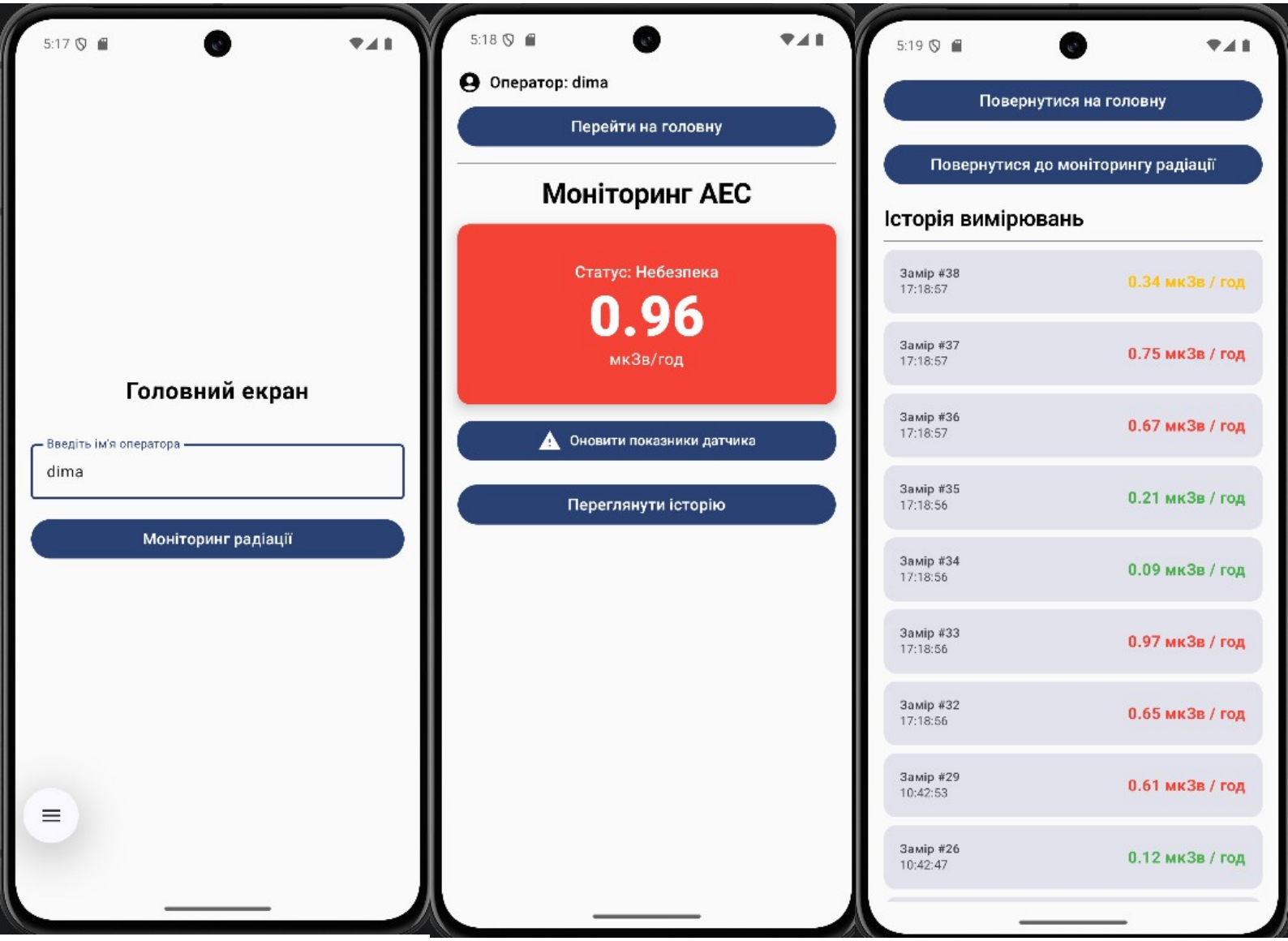


Рисунок 1.1 — Результат роботи застосунку просунутого рівня складності

Опис коду та структура

Загальна ідея

Проєкт є навчальним прикладом побудови мобільного Android-застосунку на основі Jetpack Compose з використанням бібліотеки Navigation Compose.

Структура і компоненти

Головною точкою входу виступає MainActivity, яка відповідає за ініціалізацію теми NPP_Radiation_MonitorTheme та виклику головного графа навігації Navigation(). Для організації безпечної навігації використовується sealed class Screen, у якому визначено окремі напрямки (маршрути):

1. Screen.MainScreen — головне меню із формою введення даних оператора;
2. Screen.RadiationScreen — екран онлайн-моніторингу показників датчика радіації (із підтримкою аргументу operatorName);
3. Screen.HistoryScreen — екран перегляду історії всіх збережених вимірювань.

Усі екрани побудовані як окремі, модульні Composable-функції у папці ui/screens/:

1. MainScreen — головний екран із заголовком, формою введення імені оператора (OutlinedTextField) та кнопкою для переходу до моніторингу.
2. RadiationScreen — основний екран моніторингу, де виводиться ім'я оператора (отримане через аргументи навігації), заголовок станції, картка поточного стану радіаційного фону, кнопка генерації нових показників та кнопка переходу до історії.
3. HistoryScreen — екран історії з можливістю прокрутки та кнопками повернення назад через popBackStack().

Для повторного використання елементів інтерфейсу виділено окремі компоненти в ui/components/: StatusCard — інтерактивна картка з анімованою зміною значення радіаційного фону та кольору безпеки та HistoryItem — картка окремого запису вимірювання (номер вимірювання, час, показник та колір статусу).

Використані підходи

Побудова інтерфейсу реалізована виключно засобами Jetpack Compose.

Навігація організована через Navigation Compose із застосуванням NavController, NavHost та composable.

Передача даних через аргументи: Реалізовано передачу текстових даних (operatorName) з головного екрана на екран моніторингу через шаблон маршруту radiation_screen/{operatorName} та розпарсинг аргументів з entry.arguments.

Для збереження списку історії вимірювань при переходах між екранами використано remember { mutableStateListOf<RadiationReading>() }.

Взаємодія в застосунку

Користувач запускає застосунок і потрапляє на головний екран (MainScreen). Користувач вводить ім'я в текстове поле та натискає кнопку "Моніторинг радіації". NavController здійснює перехід на RadiationScreen, передаючи введеного оператора в «URL-маршруті». На екрані моніторингу відображається передане ім'я оператора. Натискання кнопки "Оновити показники датчика" генерує нове вимірювання, анімує зміну картки StatusCard та автоматично додає нове вимірювання в спільний список історії. Натискання кнопки "Переглянути історію" здійснює перехід на HistoryScreen, де виводиться весь список вимірювань. Натискання кнопок "Повернутися..." викликає navController.popBackStack(), що повертає користувача на попередній екран із повним збереженням стану даних та імені оператора.

Підсумок

Таким чином, проєкт демонструє сучасний підхід до розробки Android-застосунків на Jetpack Compose, поєднуючи роботу аргументами навігації та управлінням реактивним станом (рисунок 1.2).

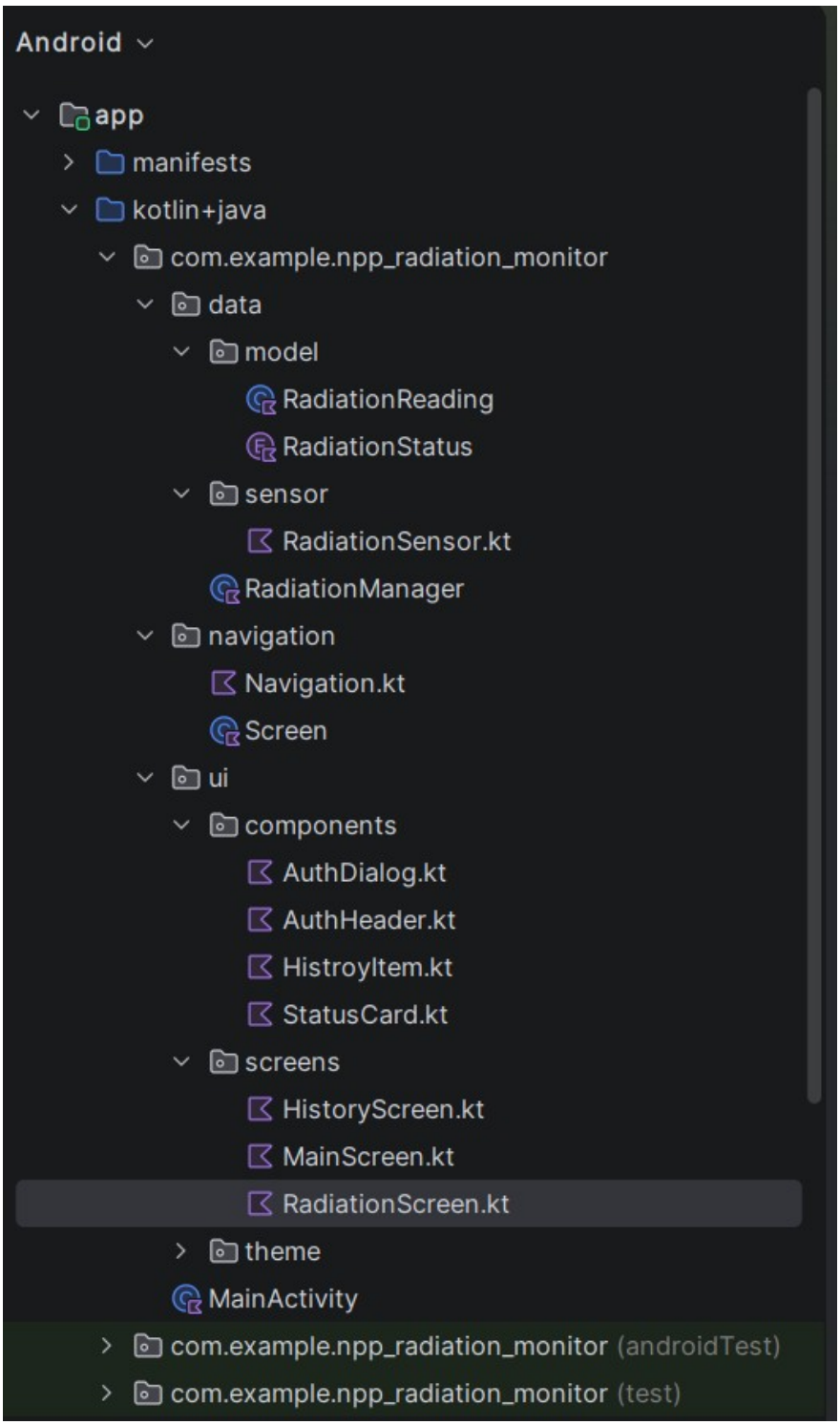


Рисунок 1.2 — Структура проєкту

ПРАКТИЧНЕ ЗАНЯТТЯ № 6

Тема: Розробка форм введення даних у мобільних застосунках з використанням Jetpack Compose, реалізація валідації введених даних та надання підказок користувачу для покращення взаємодії з UI.

Мета: Набуття практичних навичок створення інтерактивних форм, реалізації базових та розширених методів перевірки введеної інформації.

Просунутий рівень (рисунок 2.1)

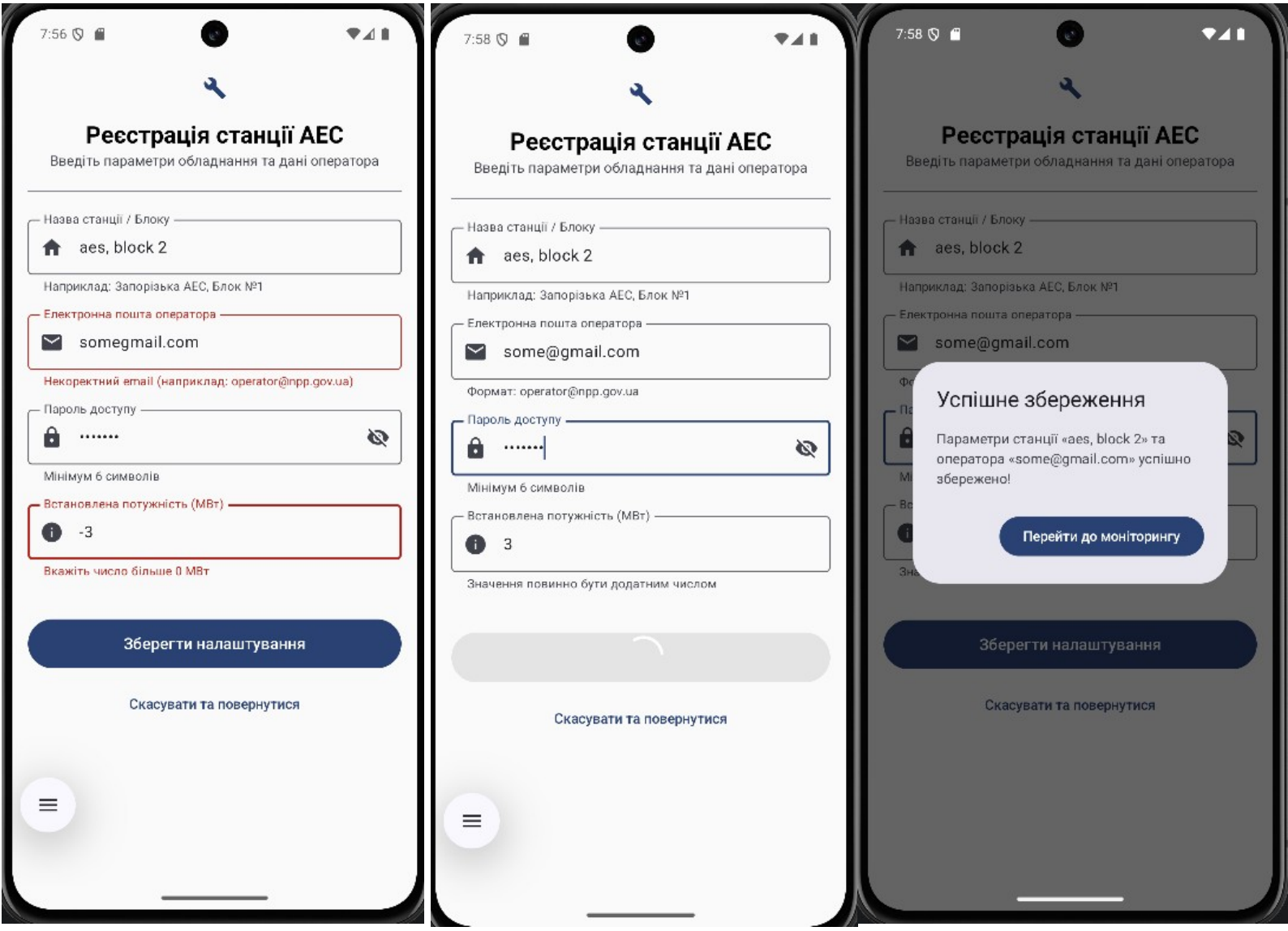


Рисунок 2.1 — Результат роботи застосунку просунутого рівня складності

Загальна ідея

Проєкт присвячений створенню інтерактивних форм введення даних у мобільному застосунку для Android з використанням Jetpack Compose. Тематика базується на прикладній задачі радіаційного моніторингу АЕС: користувач вводить інформацію про станцію (назву станції/блоку, email оператора, пароль та встановлену потужність у МВт), а система перевіряє коректність даних.

Структура і компоненти

Основною точкою входу виступає MainActivity, яка налаштовує тему застосунку та запускає головний граф навігації Navigation. Для реалізації форми створено окремий екран реєстрації станції StationFormScreen, який об'єднує модульні Composable-компоненти з папки ui/components/form/:

1. FormHeader — заголовок форми з іконкою та пояснювальним підзаголовком.
2. StationNameField — поле введення назви станції/блоку з перевіркою непорожності та підказкою.
3. EmailField — поле введення пошти оператора з перевіркою формату email через Regex та підказкою.
4. PasswordField — поле пароля з кнопкою приховування/показу символів (PasswordVisualTransformation), валідацією довжини (>= 6 символів) та підказкою.
5. PowerField — поле потужності з чисельною перевіркою (toFloatOrNull() > 0) та підказкою.
6. SubmitButton — кнопка відправки форми з вбудованим індикатором завантаження CircularProgressIndicator.

Використані підходи

Застосовано Jetpack Compose для побудови інтерфейсу та виділення кожного елемента форми в окремий Composable-компонент.

Використано механізми remember та mutableStateOf для зберігання введеного тексту, станів помилок для кожного поля та флага завантаження (isLoading).

Реалізовано перевірку обов'язкових полів на непорожність (isBlank()), відповідність формату email (Regex), мінімальну довжину пароля та перевірку чисельних значень потужності.

Використано параметр supportingText у OutlinedTextField. У разі помилки поле підсвічується червоним кольором (isError = true), а під полем виводиться зрозуміле повідомлення про помилку.

Застосовано Column, Row, Modifier.fillMaxWidth() та verticalScroll(rememberScrollState()) для коректного відображення форми на пристроях з різним розміром екранів.

Після успішного проходження валідації імітується обробка даних через LaunchedEffect з затримкою delay(1500) та показом CircularProgressIndicator.

Взаємодія в застосунку

Користувач переходить до форми через кнопку на головному екрані. Поступово заповнює поля: назву станції, email, пароль та потужність. Під кожним полем виводиться підказка (helper text). Якщо користувач вводить некоректні дані або залишає поля порожніми та натискає "Зберегти налаштування", спрацьовує функція validateForm(). Некоректні поля підсвічуються червоним кольором, а підказка змінюється на повідомлення про конкретну помилку. При заповненні всіх полів вірно, кнопка активує індикатор завантаження (CircularProgressIndicator), після чого з'являється діалог успіху AlertDialog. Після підтвердження діалогу відбувається автоматичний перехід на екран моніторингу RadiationScreen з передачею збереженої назви станції та імені оператора.

Підсумок

Проєкт демонструє побудову повноцінної, інтерактивної форми введення даних у Jetpack Compose з валідацією, динамічними підказками користувачу та станом завантаження (рисунок 2.2).

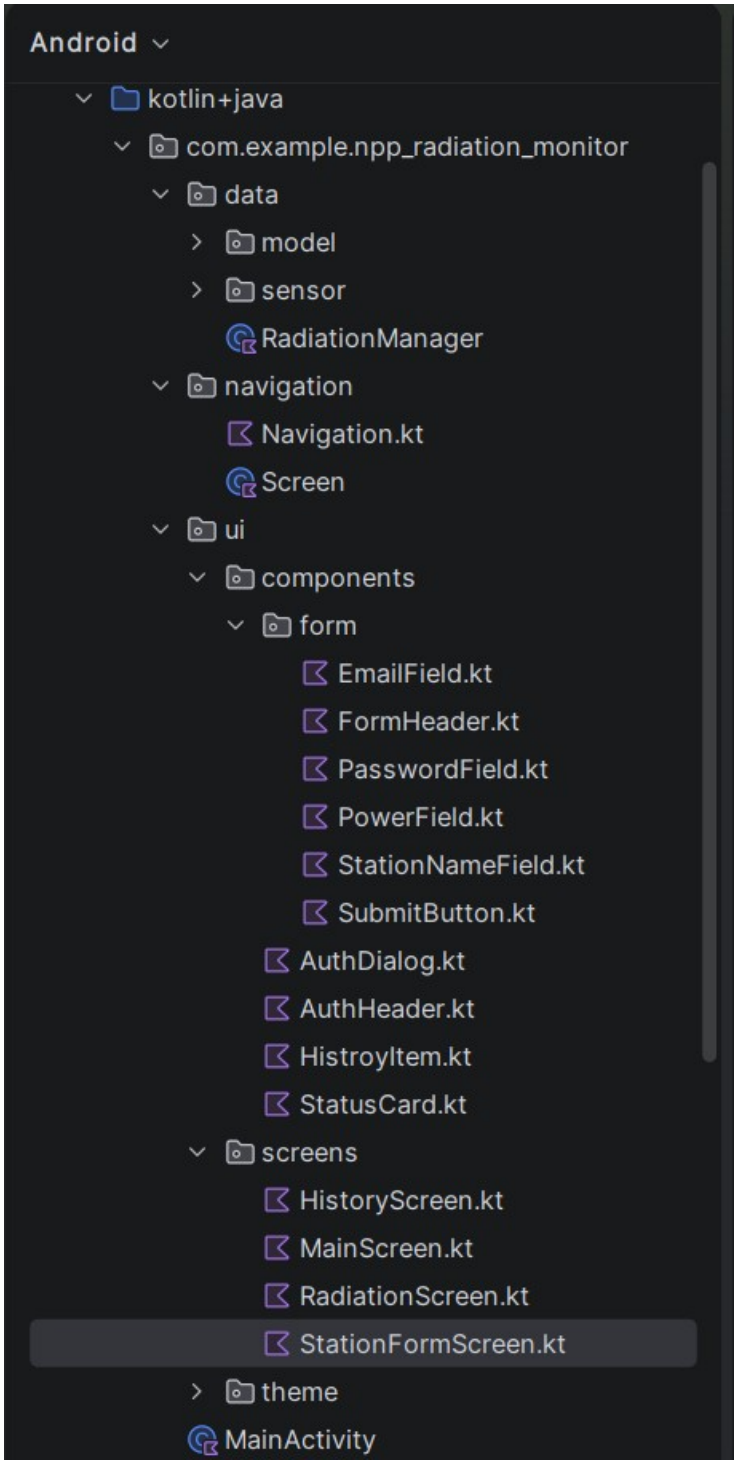


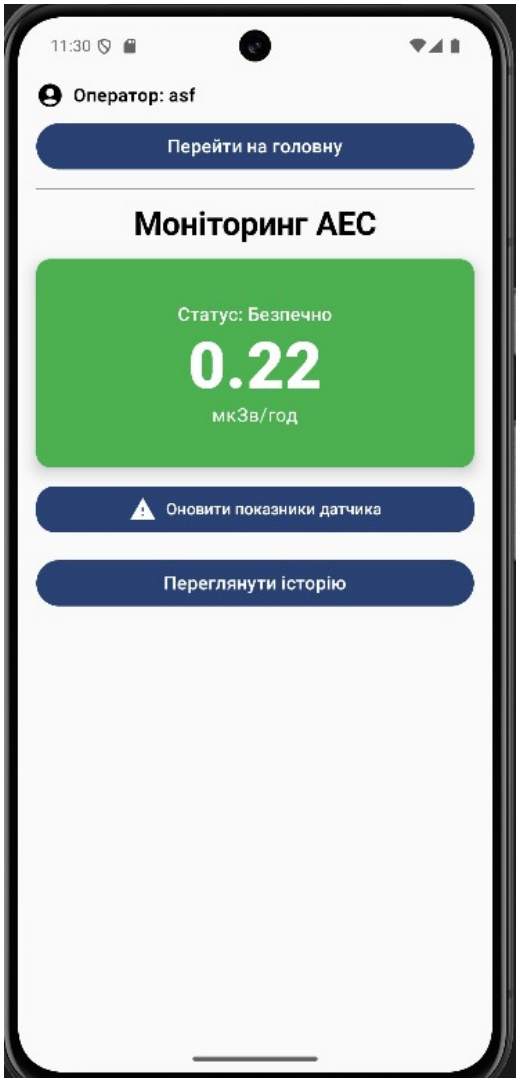
Рисунок 2.2 — Структура проєкту

ПРАКТИЧНЕ ЗАНЯТТЯ № 7

Тема: Основи архітектури MVVM у розробці мобільних застосунків на базі Kotlin і Jetpack Compose. Використання ViewModel, State, LiveData та StateFlow для управління станом і побудови «реактивного» інтерфейсу користувача.

Мета: Ознайомлення з архітектурним підходом MVVM та відпрацювання практичних навичок застосування ViewModel.

Базовий рівень (рисунок 3.1)



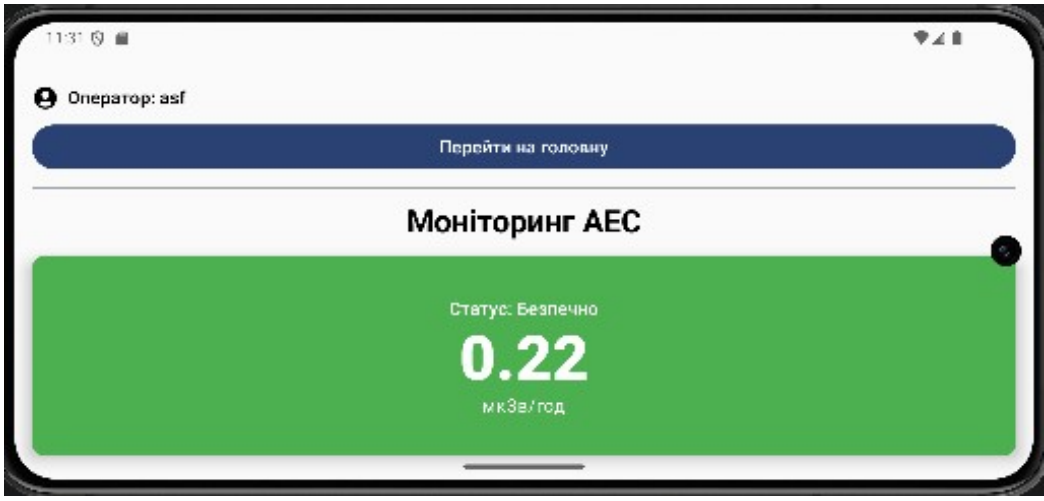


Рисунок 3.1 — Результат роботи застосунку базового рівня складності

Структура і компоненти

Основною точкою входу виступає MainActivity, яка налаштовує тему застосунку та запускає головний граф навігації Navigation. Головні компоненти архітектури MVVM у проєкту:

1. Model (Рівень даних): RadiationReading — дата-клас вимірювання (значення, час, статус). RadiationManager — менеджер для генерації показників датчика.
2. ViewModel (Рівень управління станом): RadiationViewModel — клас, що успадковується від androidx.lifecycle.ViewModel. Зберігає поточний замір currentReading у Compose State та надає метод generateNextReading() для оновлення показників.
3. View (Рівень UI): RadiationScreen — Composable-екран моніторингу, підключений до RadiationViewModel за допомогою viewModel(). StatusCard — віджет для анімованого відображення рівня радіації.

Використані підходи

Архітектура MVVM: Розділення відповідальностей між відображенням інтерфейсу (View) та управлінням станом і бізнес-логікою (ViewModel). Використання ViewModel гарантує, що згенеровані показники радіації не скидаються при повороті екрана смартфона. Реактивний Compose State: UI підключений до ViewModel і автоматично перемальовується при зміні значення currentReading. Навігація: Навігація між екранами організована за допомогою Navigation Compose (NavController та NavHost).

Взаємодія в застосунку

Користувач відкриває екран моніторингу через головне меню. RadiationScreen отримує екземпляр RadiationViewModel через виклик viewModel(). Екран зчитує поточний стан viewModel.currentReading та передає його у картку StatusCard. Натискання кнопки "Оновити показники датчика" викликає метод viewModel.generateNextReading(). ViewModel генерує нове

значення і оновлює свій стан, що змушує Compose реактивно перемалювати картку StatusCard. При повороті пристрою (зміні конфігурації) ViewModel зберігає свій стан у пам'яті, і поточні показники не втрачаються.

Підсумок

Проєкт демонструє базову реалізацію архітектури MVVM у Jetpack Compose з використанням ViewModel та State (рисунок 3.2). Завдяки відокремленню стану від UI застосунок є стійким до повороту екрана.

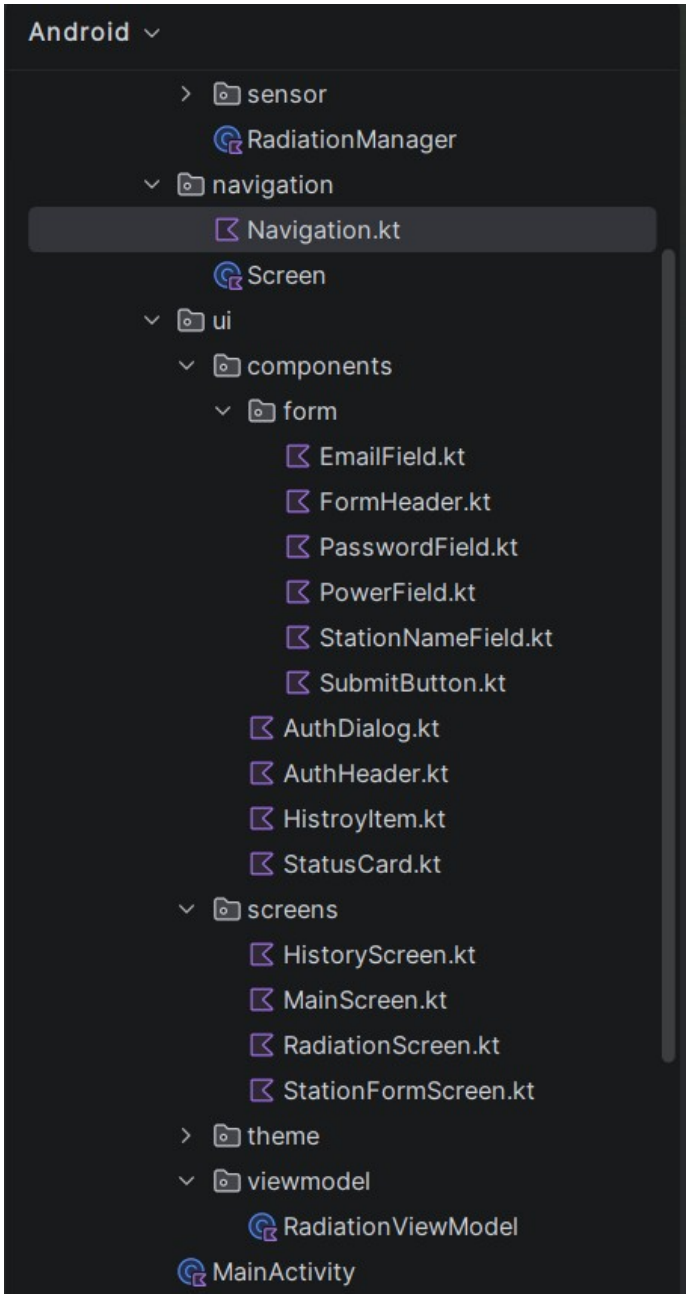


Рисунок 3.2 — Структура проєкту

ПРАКТИЧНЕ ЗАНЯТТЯ № 8

Тема: Локальне сховище даних у мобільних застосунках: використання Room Database для збереження та обробки енергетичних параметрів.

Мета: Ознайомлення з основами організації локального сховища даних у мобільних застосунках на Android.

Базовий рівень (рисунок 4.1)

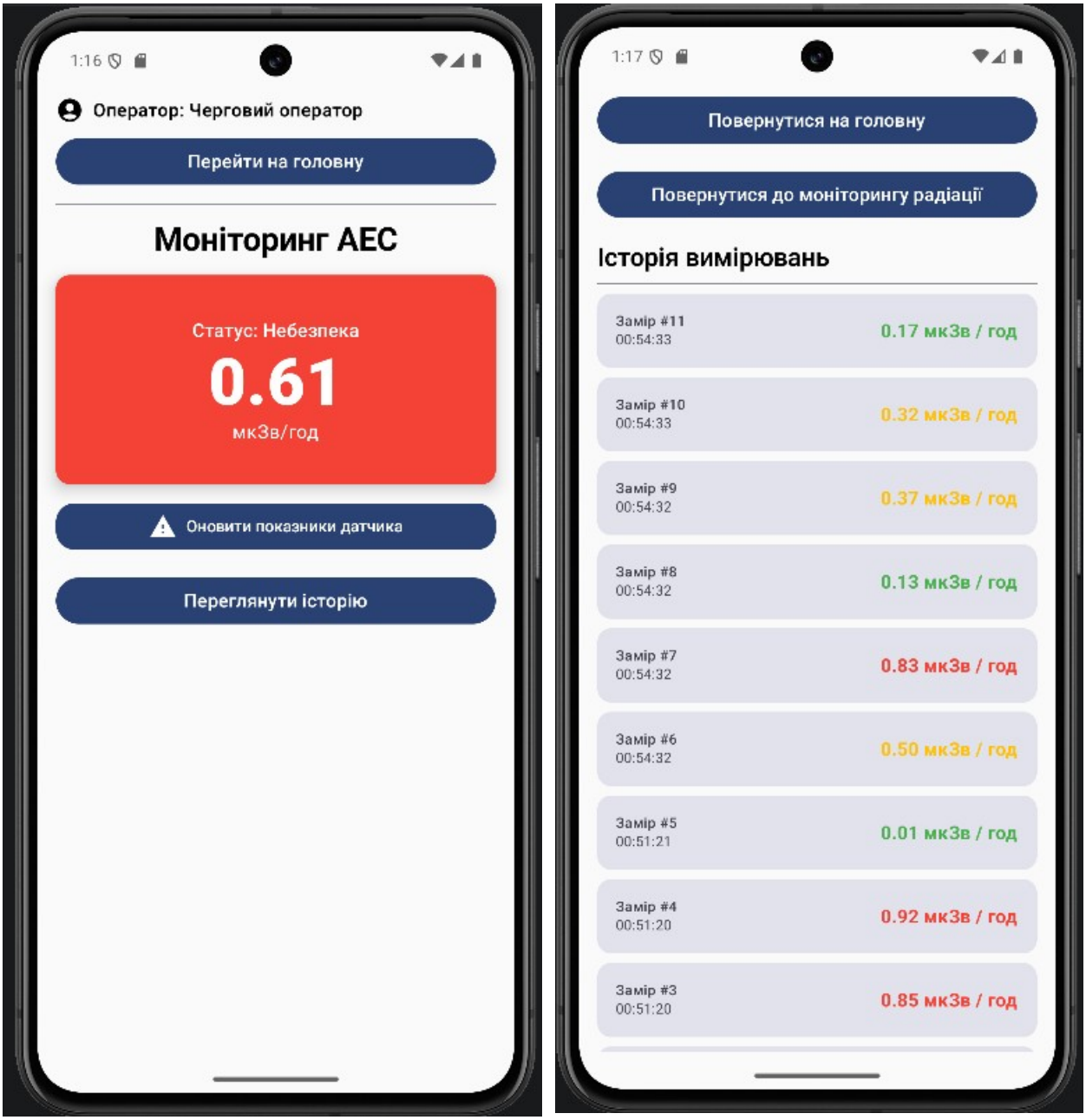


Рисунок 4.1 — Результат роботи застосунку базового рівня складності (історія зберігається навіть після перезапуску застосунку)

Загальна ідея

Проєкт присвячений організації локального сховища даних у мобільному застосунку Android за допомогою бібліотеки Room Database та паттерну Repository. Кожне нове вимірювання радіації зберігається у локальну базу даних на пристрої, а репозиторій надає реактивний потік даних (Flow) для їх відображення в UI.

Структура і компоненти

Основною точкою входу виступає MainActivity, яка налаштовує тему та запускає граф навігації Navigation. Головні компоненти архітектури Room + Repository:

1. Local Data Source (Room Database): RadiationEntity — сутність Room, що визначає структуру таблиці radiation_readings (поля id, value, timestamp, statusTitle). RadiationDao — DAO-інтерфейс з SQL-методами добавлення записів (@Insert) та отримання списку вимірювань (@Query). AppDatabase — абстрактний клас бази даних Room з підтримкою паттерну Singleton.
2. Repository Layer: RadiationRepository — клас-посередник, що надає методи для збереження вимірювань та реактивний потік allReadings.
3. ViewModel & View Layer: RadiationViewModel — звертається до RadiationRepository та асинхронно зберігає дані в Room через viewModelScope.launch. RadiationScreen — екран моніторингу з кнопкою генерування та збереження замірів.

Використані підходи

Використано анотації @Entity, @Dao, @Database, @Insert та @Query для створення таблиць.

Збереження записів у базу даних виконується у фоновому режимі через viewModelScope.launch та suspend функції.

Зчитування даних з базі відбувається через Flow<List<RadiationEntity>>, що дозволяє UI автоматично реагувати на появу нових записів.

Взаємодія в застосунку

Користувач натискає кнопку "Оновити показники датчика" на екрані моніторингу. RadiationScreen викликає метод viewModel.generateNextReading(). RadiationViewModel створює нове вимірювання та викликає метод репозиторію repository.insertReading(...). RadiationRepository через RadiationDao зберігає запис у локальну базу даних. Збережені дані залишаються на пристрої навіть після закриття чи перезапуску застосунку.

Підсумок

Проєкт демонструє побудову локального сховища даних на основі Room Database та паттерну Repository у поєднанні з MVVM і Jetpack Compose (рисунок 4.2). Завдяки Room дані надійно зберігаються на пристрої.

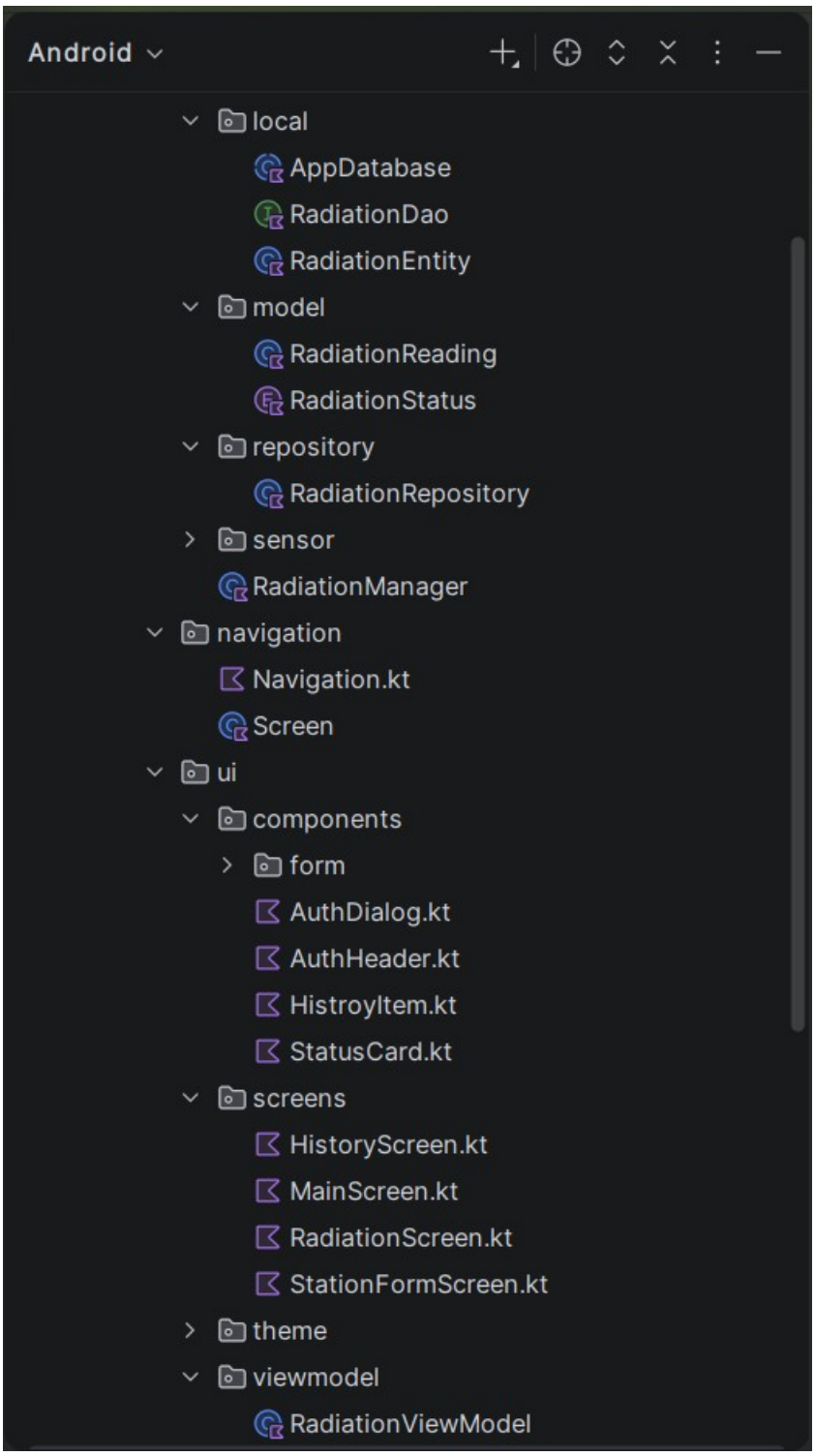


Рисунок 4.2 — Структура проєкту

ПРОГРАМНИЙ КОД ПРАКТИЧНОГО ЗАНЯТТЯ №5

Тека com/example/npp_radiation_monitor

Файл MainActivity.kt

```
package com.example.npp_radiation_monitor

import android.os.Bundle
import androidx.activity.ComponentActivity
import androidx.activity.compose.setContent
import androidx.activity.enableEdgeToEdge
import com.example.npp_radiation_monitor.navigation.Navigation
import
com.example.npp_radiation_monitor.ui.theme.NPP_Radiation_MonitorTheme

class MainActivity : ComponentActivity() {
    override fun onCreate(savedInstanceState: Bundle?) {
        super.onCreate(savedInstanceState)
        enableEdgeToEdge()
        setContent {
            NPP_Radiation_MonitorTheme {
                Navigation()
            }
        }
    }
}
```

Тека com/example/npp_radiation_monitor/data/model
Файл RadiationReading.kt

```
package com.example.npp_radiation_monitor.data.model

data class RadiationReading(
 val id: Int,
 val value: Float,
 val timestamp: String,
 val status: RadiationStatus = RadiationStatus.fromValue(value)
)
```

Тека com/example/npp_radiation_monitor/data/model
Файл RadiationStatus.kt

```
package com.example.npp_radiation_monitor.data.model

import androidx.compose.ui.graphics.Color

enum class RadiationStatus(
    val title: String,
    val color: Color,
```

```
    val maxThreshold: Float
) {
    SAFE("Безпечно", Color(0xFF4CAF50), 0.30f),
    WARNING("Увага", Color(0xFFFFC107), 0.60f),
    DANGER("Небезпека", Color(0xFFF44336), Float.MAX_VALUE);

    companion object {
        fun fromValue(value: Float): RadiationStatus {
            return when {
                value < SAFE.maxThreshold -> SAFE
                value < WARNING.maxThreshold -> WARNING
                else -> DANGER
            }
        }
    }
}
```

Тека com/example/npp_radiation_monitor/data/sensor

Файл RadiationSensor.kt

```
package com.example.npp_radiation_monitor.data.sensor

import android.os.Build
import androidx.annotation.RequiresApi
import com.example.npp_radiation_monitor.data.model.RadiationReading
import java.time.LocalDateTime
import java.time.format.DateTimeFormatter
import kotlin.random.Random

interface RadiationSensor {
    fun getReading(id: Int): RadiationReading
}

class SimulatedRadiationSensor : RadiationSensor {
    @RequiresApi(Build.VERSION_CODES.O)
    override fun getReading(id: Int): RadiationReading {
        val value = Random.nextFloat() * 1.0f // Значення від 0.00 до
1.00 мкЗв/год
        val time =
LocalDateTime.now().format(DateTimeFormatter.ofPattern("HH:mm:ss"))
        return RadiationReading(
            id = id,
            value = value,
            timestamp = time
        )
    }
}
```

Тека com/example/npp_radiation_monitor/data
Файл RadiationManager.kt

```
package com.example.npp_radiation_monitor.data

import com.example.npp_radiation_monitor.data.model.RadiationReading
import com.example.npp_radiation_monitor.data.sensor.RadiationSensor
import
com.example.npp_radiation_monitor.data.sensor.SimulatedRadiationSensor

class RadiationManager(
    private val sensor: RadiationSensor = SimulatedRadiationSensor()
) {
    private var nextId = 0

    fun createInitialReading(): RadiationReading {
        return sensor.getReading(nextId++)
    }

    fun generateNextReading(): RadiationReading {
        return sensor.getReading(nextId++)
    }
}
```

Тека com/example/npp_radiation_monitor/navigation
Файл Navigation.kt

```
package com.example.npp_radiation_monitor.navigation

import androidx.compose.runtime.Composable
import androidx.compose.runtime.mutableStateListOf
import androidx.compose.runtime.remember
import androidx.navigation.NavType
import androidx.navigation.compose.NavHost
import androidx.navigation.compose.composable
import androidx.navigation.compose.rememberNavController
import androidx.navigation.navArgument
import com.example.npp_radiation_monitor.data.RadiationManager
import com.example.npp_radiation_monitor.data.model.RadiationReading
import com.example.npp_radiation_monitor.ui.screens.HistoryScreen
import com.example.npp_radiation_monitor.ui.screens.MainScreen
import com.example.npp_radiation_monitor.ui.screens.RadiationScreen

/**
 * Головний граф навігації застосунку.
 * Управляє переходами між екранами: MainScreen, RadiationScreen та
HistoryScreen.
 * Зберігає та передає стан історії вимірювань між екранами.
 */
@Composable
fun Navigation() {
```

```
    val navController = rememberNavController()

    // Спільний стан історії вимірювань та менеджер даних
    val history = remember { mutableStateListOf<RadiationReading>() }
    val radiationManager = remember { RadiationManager() }

    NavHost(navController = navController, startDestination =
Screen.MainScreen.route) {

        composable(route = Screen.MainScreen.route) {
            MainScreen(navController = navController)
        }

        composable(
            route = Screen.RadiationScreen.route + "/{operatorName}",
            arguments = listOf(
                navArgument("operatorName") {
                    type = NavType.StringType
                }
            )
        ) { entry ->
            // Локальна змінна тільки для цього екрана
            val operatorName = entry.arguments?.getString("operatorName")
                .orEmpty()
                .ifBlank { "Default operator" }

            RadiationScreen(
                navController = navController,
                radiationManager = radiationManager,
                history = history,
                operatorName = operatorName
            )
        }

        composable(route = Screen.HistoryScreen.route) {
            HistoryScreen(navController = navController, history =
history)
        }
    }
}
```

Тека com/example/npp_radiation_monitor/navigation

Файл Screen.kt

```
package com.example.npp_radiation_monitor.navigation

sealed class Screen(val route: String) {
    object MainScreen : Screen("main_screen")
    object HistoryScreen : Screen("history_screen")
    object RadiationScreen : Screen("radiation_screen") {
```

```
        fun createRoute(operatorName: String): String {
            return "radiation_screen/$operatorName"
        }
    }
}
```

Тека com/example/npp_radiation_monitor/ui/components
Файл AuthDialog.kt

```
package com.example.npp_radiation_monitor.ui.components

import androidx.compose.foundation.layout.Column
import androidx.compose.foundation.layout.Spacer
import androidx.compose.foundation.layout.fillMaxWidth
import androidx.compose.foundation.layout.height
import androidx.compose.foundation.text.KeyboardOptions
import androidx.compose.material.icons.Icons
import androidx.compose.material.icons.filled.Email
import androidx.compose.material.icons.filled.Lock
import androidx.compose.material.icons.filled.Person
import androidx.compose.material.icons.filled.Visibility
import androidx.compose.material.icons.filled.VisibilityOff
import androidx.compose.material3.AlertDialog
import androidx.compose.material3.Button
import androidx.compose.material3.Icon
import androidx.compose.material3.IconButton
import androidx.compose.material3.MaterialTheme
import androidx.compose.material3.OutlinedTextField
import androidx.compose.material3.Text
import androidx.compose.material3.TextButton
import androidx.compose.runtime.Composable
import androidx.compose.runtime.getValue
import androidx.compose.runtime.mutableStateOf
import androidx.compose.runtime.remember
import androidx.compose.runtime.setValue
import androidx.compose.ui.Modifier
import androidx.compose.ui.graphics.Color
import androidx.compose.ui.text.font.FontWeight
import androidx.compose.ui.text.input.KeyboardType
import androidx.compose.ui.text.input.PasswordVisualTransformation
import androidx.compose.ui.text.input.VisualTransformation
import androidx.compose.ui.unit.dp

fun checkErrors(email: String, password: String): String? {
    return when {
        email.isBlank() -> "Заповніть електронну пошту"
        password.length < 6 -> "Пароль повинен містити більше 6 символів"
        else -> null
    }
}
```

```
@Composable
fun Errors(error: String?) {
    if (error == null) return
    if (error.isNotEmpty()) {
        Text(
            text = error,
            color = Color.Red,
            fontWeight = FontWeight.Bold
        )
    }
}

@Composable
fun AuthDialog(
    onDismissRequest: () -> Unit,
    onSuccess: (operatorName: String) -> Unit
) {
    // Перемикач режимів: false = вхід, true = реєстрація
    var isRegisterMode by remember { mutableStateOf(false) }

    // Поля введення
    var email by remember { mutableStateOf("") }
    var password by remember { mutableStateOf("") }
    var name by remember { mutableStateOf("") }
    var passwordVisible by remember { mutableStateOf(false) }

    // Булева змінна, яка каже, чи показувати помилку, чи ні
    var showError by remember { mutableStateOf(false) }

    AlertDialog(
        onDismissRequest = onDismissRequest,
        title = {
            Text(
                text = if (isRegisterMode) "Реєстрація оператора АЕС"
else "Вхід у систему АЕС",
                style = MaterialTheme.typography.titleLarge,
                fontWeight = FontWeight.Bold
            )
        },
        text = {
            Column(modifier = Modifier.fillMaxWidth()) {
                // Поле імені (показується тільки при реєстрації)
                if (isRegisterMode) {
                    OutlinedTextField(
                        value = name,
                        onValueChange = { name = it; showError = false },
                        label = { Text("ПІБ Оператора") },
                        leadingIcon = { Icon(Icons.Default.Person,
contentDescription = null) },
                        singleLine = true,
```

```
                        modifier = Modifier.fillMaxWidth()
                    )
                    Spacer(modifier = Modifier.height(8.dp))
                }

                // Поле email
                OutlinedTextField(
                    value = email,
                    onValueChange = { email = it; showError = false },
                    label = { Text("Електронна пошта") },
                    leadingIcon = { Icon(Icons.Default.Email,
contentDescription = null) },
                    keyboardOptions = KeyboardOptions(keyboardType =
KeyboardType.Email),
                    singleLine = true,
                    modifier = Modifier.fillMaxWidth()
                )

                Spacer(modifier = Modifier.height(8.dp))

                // Поле password
                OutlinedTextField(
                    value = password,
                    onValueChange = { password = it; showError = false },
                    label = { Text("Пароль") },
                    leadingIcon = { Icon(Icons.Default.Lock,
contentDescription = null) },
                    trailingIcon = {
                        IconButton(onClick = { passwordVisible = !
passwordVisible }) {
                            Icon(
                                imageVector = if (passwordVisible)
Icons.Default.Visibility else Icons.Default.VisibilityOff,
                                contentDescription = null
                            )
                        }
                    },
                    visualTransformation = if (passwordVisible)
VisualTransformation.None else PasswordVisualTransformation(),
                    keyboardOptions = KeyboardOptions(keyboardType =
KeyboardType.Password),
                    singleLine = true,
                    modifier = Modifier.fillMaxWidth()
                )

                Spacer(modifier = Modifier.height(8.dp))

                if (showError) Errors(checkErrors(email, password))

                Spacer(modifier=Modifier.height(8.dp))
```

```
                // Кнопка перемикання вхід / реєстрація
                TextButton(
                    onClick = { isRegisterMode = !isRegisterMode;
showError = false },
                    modifier = Modifier.fillMaxWidth()
                ) {
                    Text(
                        text = if (isRegisterMode) "Вже є акаунт? Увійти"
else "Немає акаунту? Зареєструватися"
                    )
                }
            }
        },
        confirmButton = {
            Button(
                onClick = {
                    // Перевіряємо, чи є помилки
                    if (checkErrors(email, password) != null) {
                        showError = true
                    } else {
                        onSuccess(email)
                    }
                }
            ) {
                Text(if (isRegisterMode) "Зареєструватися" else "Увійти")
            }
        },
        dismissButton = {
            TextButton(onClick = onDismissRequest) {
                Text("Скасувати")
            }
        }
    )
}
```

Тека com/example/npp_radiation_monitor/ui/components

Файл AuthHeader.kt

```
package com.example.npp_radiation_monitor.ui.components

import androidx.compose.foundation.layout.Arrangement
import androidx.compose.foundation.layout.Row
import androidx.compose.foundation.layout.Spacer
import androidx.compose.foundation.layout.fillMaxWidth
import androidx.compose.foundation.layout.width
import androidx.compose.material.icons.Icons
import androidx.compose.material.icons.automirrored.filled.ExitToApp
import androidx.compose.material.icons.filled.AccountCircle
import androidx.compose.material3.Icon
import androidx.compose.material3.MaterialTheme
```

```
import androidx.compose.material3.OutlinedButton
import androidx.compose.material3.Text
import androidx.compose.material3.TextButton
import androidx.compose.runtime.Composable
import androidx.compose.ui.Alignment
import androidx.compose.ui.Modifier
import androidx.compose.ui.text.font.FontWeight
import androidx.compose.ui.unit.dp

@Composable
fun AuthHeader(operatorEmail: String?, onAuthClick: () -> Unit,
onLogoutClick: () -> Unit) {
    Row(
            modifier = Modifier.fillMaxWidth(),
            horizontalArrangement = Arrangement.SpaceBetween,
            verticalAlignment = Alignment.CenterVertically
        ) {
            Row(verticalAlignment = Alignment.CenterVertically) {
                Icon(
                    imageVector = Icons.Default.AccountCircle,
                    contentDescription = "Профіль",
                    tint = MaterialTheme.colorScheme.primary
                )
                Spacer(modifier = Modifier.width(8.dp))
                Text(
                    text = if (operatorEmail != null) "Оператор:
$operatorEmail" else "Статус: Гість",
                    style = MaterialTheme.typography.bodyMedium,
                    fontWeight = FontWeight.SemiBold
                )
            }

            if (operatorEmail == null) {
                OutlinedButton(onClick = onAuthClick) {
                    Text("Увійти")
                }
            } else {
                TextButton(onClick = onLogoutClick) {
                    Icon(Icons.AutoMirrored.Filled.ExitToApp,
contentDescription = null)
                    Spacer(modifier = Modifier.width(4.dp))
                    Text("Вийти")
                }
            }
        }
}
```

Тека com/example/npp_radiation_monitor/ui/components
Файл HistoryItem.kt

```
package com.example.npp_radiation_monitor.ui.components

import androidx.compose.foundation.layout.Arrangement
import androidx.compose.foundation.layout.Column
import androidx.compose.foundation.layout.Row
import androidx.compose.foundation.layout.fillMaxWidth
import androidx.compose.foundation.layout.padding
import androidx.compose.material3.Card
import androidx.compose.material3.CardDefaults
import androidx.compose.material3.MaterialTheme
import androidx.compose.material3.Text
import androidx.compose.runtime.Composable
import androidx.compose.ui.Alignment
import androidx.compose.ui.Modifier
import androidx.compose.ui.text.font.FontWeight
import androidx.compose.ui.unit.dp
import com.example.npp_radiation_monitor.data.model.RadiationReading
import java.util.Locale

/**
 * Окремий елемент списку історії вимірювань (HistoryItem).
 * Відображає номер вимірювання, час проведення та показник радіації з
кольором статусу.
 */
@Composable
fun HistoryItem(reading: RadiationReading) {
    Card(
        modifier = Modifier.fillMaxWidth(),
        colors = CardDefaults.cardColors(containerColor =
MaterialTheme.colorScheme.surfaceVariant)
    ) {
        Row(
            modifier = Modifier
                .padding(16.dp)
                .fillMaxWidth(),
            horizontalArrangement = Arrangement.SpaceBetween,
            verticalAlignment = Alignment.CenterVertically
        ) {
            Column {
                Text(text = "Замір #${reading.id}", style =
MaterialTheme.typography.labelMedium)
                Text(text = reading.timestamp, style =
MaterialTheme.typography.bodySmall)
            }
            Text(
                text = String.format(locale = Locale.UK, "%.2f мкЗв /
год", reading.value),
                style = MaterialTheme.typography.titleMedium,
```

```
                fontWeight = FontWeight.Bold,
                color = reading.status.color
            )
        }
    }
}
```

Тека com/example/npp_radiation_monitor/ui/components
Файл StatusCard.kt

```
package com.example.npp_radiation_monitor.ui.components

import androidx.compose.animation.animateColorAsState
import androidx.compose.animation.core.animateFloatAsState
import androidx.compose.foundation.layout.Arrangement
import androidx.compose.foundation.layout.Column
import androidx.compose.foundation.layout.fillMaxSize
import androidx.compose.foundation.layout.fillMaxWidth
import androidx.compose.foundation.layout.height
import androidx.compose.material3.Card
import androidx.compose.material3.CardDefaults
import androidx.compose.material3.MaterialTheme
import androidx.compose.material3.Text
import androidx.compose.runtime.Composable
import androidx.compose.runtime.getValue
import androidx.compose.ui.Alignment
import androidx.compose.ui.Modifier
import androidx.compose.ui.graphics.Color
import androidx.compose.ui.text.font.FontWeight
import androidx.compose.ui.unit.dp
import androidx.compose.ui.unit.sp
import com.example.npp_radiation_monitor.data.model.RadiationReading
import java.util.Locale

/**
 * Компонент картки статусу радіаційного фону (StatusCard).
 * Відображає поточний рівень радіації із анімованим зміненням кольору
 * та значення залежно від статусу безпеки.
 */
@Composable
fun StatusCard(
    reading: RadiationReading,
    modifier: Modifier = Modifier
) {
    // Анімації значення радіації та кольору картки
    val animatedRadiation by animateFloatAsState(targetValue =
reading.value)
    val animatedColor by animateColorAsState(targetValue =
reading.status.color)
```

```
    Card(
        modifier = modifier
            .fillMaxWidth()
            .height(180.dp),
        colors = CardDefaults.cardColors(containerColor = animatedColor),
        elevation = CardDefaults.cardElevation(defaultElevation = 8.dp)
    ) {
        Column(
            modifier = Modifier.fillMaxSize(),
            verticalArrangement = Arrangement.Center,
            horizontalAlignment = Alignment.CenterHorizontally
        ) {
            // Текст статусу (Норма, Увага, Небезпека)
            Text(
                text = "Статус: ${reading.status.title}",
                color = Color.White,
                style = MaterialTheme.typography.titleMedium
            )
            // Анімоване числове значення радіації
            Text(
                text = String.format(locale = Locale.UK, "%.2f",
animatedRadiation),
                color = Color.White,
                fontSize = 56.sp,
                fontWeight = FontWeight.Black
            )
            // Одиниці вимірювання
            Text(
                text = "мкЗв/год",
                color = Color.White,
                style = MaterialTheme.typography.bodyLarge
            )
        }
    }
}
```

Тека com/example/npp_radiation_monitor/ui/screens
Файл HistoryScreen.kt

```
package com.example.npp_radiation_monitor.ui.screens

import androidx.compose.foundation.layout.Arrangement
import androidx.compose.foundation.layout.Column
import androidx.compose.foundation.layout.Row
import androidx.compose.foundation.layout.Spacer
import androidx.compose.foundation.layout.fillMaxWidth
import androidx.compose.foundation.layout.padding
import androidx.compose.foundation.lazy.LazyColumn
import androidx.compose.foundation.lazy.items
```

```
import androidx.compose.material3.Button
import androidx.compose.material3.HorizontalDivider
import androidx.compose.material3.MaterialTheme
import androidx.compose.material3.Text
import androidx.compose.runtime.Composable
import androidx.compose.ui.Alignment
import androidx.compose.ui.Modifier
import androidx.compose.ui.text.font.FontWeight
import androidx.compose.ui.unit.dp
import androidx.compose.ui.unit.sp
import androidx.navigation.NavController
import com.example.npp_radiation_monitor.data.model.RadiationReading
import com.example.npp_radiation_monitor.navigation.Screen
import com.example.npp_radiation_monitor.ui.components.HistoryItem

/**
 * Екран історії вимірювань (HistoryScreen).
 * Відображає список усіх збережених вимірювань радіаційного фону.
 */
@Composable
fun HistoryScreen(
    navController: NavController,
    history: List<RadiationReading>
) {
    Column(
        modifier = Modifier
            .fillMaxWidth()
            .padding(16.dp)
            .padding(top = 40.dp),
        horizontalAlignment = Alignment.CenterHorizontally
    ) {
        // Кнопка повернення на головний екран
        Button(
            onClick =
{ navController.navigate(Screen.MainScreen.route) },
            modifier = Modifier.fillMaxWidth()
        ) {
            Text(text = "Повернутися на головну", fontSize = 16.sp)
        }

        Spacer(modifier = Modifier.padding(8.dp))

        // Кнопка повернення на екран моніторингу
        Button(
            onClick = { navController.popBackStack() },
            modifier = Modifier.fillMaxWidth()
        ) {
            Text(text = "Повернутися до моніторингу радіації", fontSize =
16.sp)
        }
```

```
        Spacer(modifier = Modifier.padding(8.dp))

        // Заголовок екрана історії
        Row(
            modifier = Modifier.fillMaxWidth(),
            horizontalArrangement = Arrangement.Start
        ) {
            Text(
                text = "Історія вимірювань",
                style = MaterialTheme.typography.titleLarge,
                fontWeight = FontWeight.SemiBold
            )
        }

        HorizontalDivider(modifier = Modifier.padding(vertical = 8.dp))

        // Список усіх вимірювань
        LazyColumn(
            modifier = Modifier
                .fillMaxWidth()
                .padding(bottom = 16.dp),
            verticalArrangement = Arrangement.spacedBy(8.dp)
        ) {
            items(history) { item ->
                HistoryItem(reading = item)
            }
        }
    }
}
```

Тека com/example/npp_radiation_monitor/ui/screens

Файл MainScreen.kt

```
package com.example.npp_radiation_monitor.ui.screens

import androidx.compose.foundation.layout.Arrangement
import androidx.compose.foundation.layout.Column
import androidx.compose.foundation.layout.Spacer
import androidx.compose.foundation.layout.fillMaxHeight
import androidx.compose.foundation.layout.fillMaxWidth
import androidx.compose.foundation.layout.height
import androidx.compose.foundation.layout.padding
import androidx.compose.material3.Button
import androidx.compose.material3.OutlinedTextField
import androidx.compose.material3.Text
import androidx.compose.runtime.Composable
import androidx.compose.runtime.getValue
import androidx.compose.runtime.mutableStateOf
import androidx.compose.runtime.remember
import androidx.compose.runtime.setValue
```

```
import androidx.compose.ui.Alignment
import androidx.compose.ui.Modifier
import androidx.compose.ui.text.font.FontWeight
import androidx.compose.ui.unit.dp
import androidx.compose.ui.unit.sp
import androidx.navigation.NavController
import com.example.npp_radiation_monitor.navigation.Screen

/**
 * Головний екран застосунку (MainScreen).
 * Містить форму введення імені оператора та кнопку для переходу
 * до моніторингу радіації з передачею даних через аргументи навігації.
 */
@Composable
fun MainScreen(navController: NavController) {
    // Стан для збереження введеного імені оператора
    var operatorName by remember { mutableStateOf("") }

    Column(
        modifier = Modifier
            .fillMaxWidth()
            .fillMaxHeight()
            .padding(16.dp),
        horizontalAlignment = Alignment.CenterHorizontally,
        verticalArrangement = Arrangement.Center
    ) {
        // Заголовок екрана
        Text(
            text = "Головний екран",
            fontSize = 24.sp,
            fontWeight = FontWeight.Bold
        )

        Spacer(modifier = Modifier.padding(16.dp))

        // Форма введення імені оператора
        OutlinedTextField(
            value = operatorName,
            onValueChange = { operatorName = it },
            label = { Text("Введіть ім'я оператора") },
            modifier = Modifier.fillMaxWidth(),
            singleLine = true
        )

        Spacer(modifier = Modifier.height(16.dp))

        // Кнопка переходу до екрана моніторингу
        Button(
            modifier = Modifier.fillMaxWidth(),
            onClick = {
```

```
navController.navigate(Screen.RadiationScreen.createRoute(operatorName))
            }
        ) {
            Text(text = "Моніторинг радіації", fontSize = 16.sp)
        }
    }
}
```

Тека com/example/npp_radiation_monitor/ui/screens
Файл RadiationScreen.kt

```
package com.example.npp_radiation_monitor.ui.screens

import androidx.compose.foundation.layout.Column
import androidx.compose.foundation.layout.Row
import androidx.compose.foundation.layout.Spacer
import androidx.compose.foundation.layout.fillMaxSize
import androidx.compose.foundation.layout.fillMaxWidth
import androidx.compose.foundation.layout.height
import androidx.compose.foundation.layout.padding
import androidx.compose.foundation.layout.width
import androidx.compose.material.icons.Icons
import androidx.compose.material.icons.filled.AccountCircle
import androidx.compose.material.icons.filled.Warning
import androidx.compose.material3.Button
import androidx.compose.material3.HorizontalDivider
import androidx.compose.material3.Icon
import androidx.compose.material3.MaterialTheme
import androidx.compose.material3.Text
import androidx.compose.runtime.Composable
import androidx.compose.runtime.getValue
import androidx.compose.runtime.mutableStateOf
import androidx.compose.runtime.remember
import androidx.compose.runtime.setValue
import androidx.compose.runtime.snapshots.SnapshotStateList
import androidx.compose.ui.Alignment
import androidx.compose.ui.Modifier
import androidx.compose.ui.text.font.FontWeight
import androidx.compose.ui.unit.dp
import androidx.compose.ui.unit.sp
import androidx.navigation.NavController
import com.example.npp_radiation_monitor.data.RadiationManager
import com.example.npp_radiation_monitor.data.model.RadiationReading
import com.example.npp_radiation_monitor.navigation.Screen
import com.example.npp_radiation_monitor.ui.components.StatusCard

/**
 * Екран моніторингу радіації (RadiationScreen).
 * Відображає поточні показники радіаційного фону, дозволяє генерувати
```

```
 * нові вимірювання та зберігає їх у спільну історію.
 */
@Composable
fun RadiationScreen(
    navController: NavController,
    radiationManager: RadiationManager,
    history: SnapshotStateList<RadiationReading>,
    operatorName: String
) {
    // Поточне вимірювання датчика
    var currentReading by remember
{ mutableStateOf(radiationManager.createInitialReading()) }

    Column(
        modifier = Modifier
            .fillMaxSize()
            .padding(16.dp)
            .padding(top = 40.dp),
        horizontalAlignment = Alignment.CenterHorizontally,
    ) {
        // Рядок із інформацією про оператора
        Row(
            modifier = Modifier
                .fillMaxWidth()
                .padding(bottom = 8.dp),
            verticalAlignment = Alignment.CenterVertically
        ) {
            Icon(Icons.Default.AccountCircle, contentDescription = null)
            Spacer(modifier = Modifier.width(8.dp))
            Text(
                text = "Оператор: $operatorName",
                style = MaterialTheme.typography.titleMedium,
                fontWeight = FontWeight.SemiBold
            )
        }

        // Кнопка повернення на головну
        Button(
            onClick =
{ navController.navigate(Screen.MainScreen.route) },
            modifier = Modifier.fillMaxWidth()
        ) {
            Text(text = "Перейти на головну", fontSize = 16.sp)
        }

        HorizontalDivider(modifier = Modifier.padding(vertical = 12.dp))

        // Заголовок станції
        Text(
            text = "Моніторинг АЕС",
            style = MaterialTheme.typography.headlineMedium,
```

```
            fontWeight = FontWeight.Bold,
            modifier = Modifier.padding(bottom = 12.dp)
        )

        // Картка поточного стану радіації
        StatusCard(reading = currentReading)

        Spacer(modifier = Modifier.height(12.dp))

        // Кнопка оновлення показників датчика
        Button(
            onClick = {
                val newReading = radiationManager.generateNextReading()
                currentReading = newReading
                history.add(0, newReading)
            },
            modifier = Modifier.fillMaxWidth(),
            shape = MaterialTheme.shapes.large
        ) {
            Icon(Icons.Default.Warning, contentDescription = null)
            Spacer(Modifier.width(8.dp))
            Text("Оновити показники датчика")
        }

        Spacer(modifier = Modifier.height(16.dp))

        // Кнопка переходу до екрана історії
        Button(
            onClick =
{ navController.navigate(Screen.HistoryScreen.route) },
            modifier = Modifier.fillMaxWidth()
        ) {
            Text(text = "Переглянути історію", fontSize = 16.sp)
        }
    }
}
```

ПРОГРАМНИЙ КОД ПРАКТИЧНОГО ЗАНЯТТЯ №6

Тека com/example/npp_radiation_monitor

Файл MainActivity.kt

```
package com.example.npp_radiation_monitor

import android.os.Bundle
import androidx.activity.ComponentActivity
import androidx.activity.compose.setContent
import androidx.activity.enableEdgeToEdge
import com.example.npp_radiation_monitor.navigation.Navigation
import
com.example.npp_radiation_monitor.ui.theme.NPP_Radiation_MonitorTheme

class MainActivity : ComponentActivity() {
    override fun onCreate(savedInstanceState: Bundle?) {
        super.onCreate(savedInstanceState)
        enableEdgeToEdge()
        setContent {
            NPP_Radiation_MonitorTheme {
                Navigation()
            }
        }
    }
}
```

Тека com/example/npp_radiation_monitor/data/model

Файл RadiationReading.kt

```
package com.example.npp_radiation_monitor.data.model

data class RadiationReading(
 val id: Int,
 val value: Float,
 val timestamp: String,
 val status: RadiationStatus = RadiationStatus.fromValue(value)
)
```

Тека com/example/npp_radiation_monitor/data/model

Файл RadiationStatus.kt

```
package com.example.npp_radiation_monitor.data.model

import androidx.compose.ui.graphics.Color

enum class RadiationStatus(
    val title: String,
    val color: Color,
```

```
    val maxThreshold: Float
) {
    SAFE("Безпечно", Color(0xFF4CAF50), 0.30f),
    WARNING("Увага", Color(0xFFFFC107), 0.60f),
    DANGER("Небезпека", Color(0xFFF44336), Float.MAX_VALUE);

    companion object {
        fun fromValue(value: Float): RadiationStatus {
            return when {
                value < SAFE.maxThreshold -> SAFE
                value < WARNING.maxThreshold -> WARNING
                else -> DANGER
            }
        }
    }
}
```

Тека com/example/npp_radiation_monitor/data/sensor

Файл RadiationSensor.kt

```
package com.example.npp_radiation_monitor.data.sensor

import android.os.Build
import androidx.annotation.RequiresApi
import com.example.npp_radiation_monitor.data.model.RadiationReading
import java.time.LocalDateTime
import java.time.format.DateTimeFormatter
import kotlin.random.Random

interface RadiationSensor {
    fun getReading(id: Int): RadiationReading
}

class SimulatedRadiationSensor : RadiationSensor {
    @RequiresApi(Build.VERSION_CODES.O)
    override fun getReading(id: Int): RadiationReading {
        val value = Random.nextFloat() * 1.0f // Значення від 0.00 до
1.00 мкЗв/год
        val time =
LocalDateTime.now().format(DateTimeFormatter.ofPattern("HH:mm:ss"))
        return RadiationReading(
            id = id,
            value = value,
            timestamp = time
        )
    }
}
```

Тека com/example/npp_radiation_monitor/data
Файл RadiationManager.kt

```
package com.example.npp_radiation_monitor.data

import com.example.npp_radiation_monitor.data.model.RadiationReading
import com.example.npp_radiation_monitor.data.sensor.RadiationSensor
import
com.example.npp_radiation_monitor.data.sensor.SimulatedRadiationSensor

class RadiationManager(
    private val sensor: RadiationSensor = SimulatedRadiationSensor()
) {
    private var nextId = 0

    fun createInitialReading(): RadiationReading {
        return sensor.getReading(nextId++)
    }

    fun generateNextReading(): RadiationReading {
        return sensor.getReading(nextId++)
    }
}
```

Тека com/example/npp_radiation_monitor/navigation
Файл Navigation.kt

```
package com.example.npp_radiation_monitor.navigation

import androidx.compose.runtime.Composable
import androidx.compose.runtime.mutableStateListOf
import androidx.compose.runtime.remember
import androidx.navigation.NavType
import androidx.navigation.compose.NavHost
import androidx.navigation.compose.composable
import androidx.navigation.compose.rememberNavController
import androidx.navigation.navArgument
import com.example.npp_radiation_monitor.data.RadiationManager
import com.example.npp_radiation_monitor.data.model.RadiationReading
import com.example.npp_radiation_monitor.ui.screens.HistoryScreen
import com.example.npp_radiation_monitor.ui.screens.MainScreen
import com.example.npp_radiation_monitor.ui.screens.RadiationScreen
import com.example.npp_radiation_monitor.ui.screens.StationFormScreen

/**
 * Головний граф навігації застосунку.
 * Управляє переходами між екранами: MainScreen, RadiationScreen,
HistoryScreen та StationFormScreen.
 * Зберігає та передає стан історії вимірювань між екранами.
 */
```

```
@Composable
fun Navigation() {
    val navController = rememberNavController()

    // Спільний стан історії вимірювань та менеджер даних
    val history = remember { mutableStateListOf<RadiationReading>() }
    val radiationManager = remember { RadiationManager() }

    NavHost(navController = navController, startDestination =
Screen.MainScreen.route) {

        composable(route = Screen.MainScreen.route) {
            MainScreen(navController = navController)
        }

        composable(route = Screen.FormScreen.route) {
            StationFormScreen(navController = navController)
        }

        composable(
            route = Screen.RadiationScreen.route +
"/{stationName}/{operatorName}",
            arguments = listOf(
                navArgument("stationName") {
                    type = NavType.StringType
                    defaultValue = "Моніторинг АЕС"
                },
                navArgument("operatorName") {
                    type = NavType.StringType
                    defaultValue = "Черговий оператор"
                }
            )
        ) { entry ->
            // Отримуємо і назву станції, і ім'я оператора з аргументів
маршруту
            val stationName = entry.arguments?.getString("stationName")
                .orEmpty()
                .ifBlank { "Моніторинг АЕС" }
            val operatorName = entry.arguments?.getString("operatorName")
                .orEmpty()
                .ifBlank { "Черговий оператор" }

            RadiationScreen(
                navController = navController,
                radiationManager = radiationManager,
                history = history,
                operatorName = operatorName,
                stationName = stationName
            )
        }
```

```
        composable(route = Screen.HistoryScreen.route) {
            HistoryScreen(navController = navController, history =
history)
        }
    }
}
```

Тека com/example/npp_radiation_monitor/navigation
Файл Screen.kt

```
package com.example.npp_radiation_monitor.navigation

sealed class Screen(val route: String) {
    object MainScreen : Screen("main_screen")
    object HistoryScreen : Screen("history_screen")
    object FormScreen : Screen("form_screen")
    object RadiationScreen : Screen("radiation_screen") {
        fun createRoute(
            stationName: String = "Моніторинг АЕС",
            operatorName: String = "Черговий оператор"
        ): String {
            val safeStation = stationName.ifBlank { "Моніторинг АЕС" }
            val safeOperator = operatorName.ifBlank { "Черговий оператор"
}
            return "radiation_screen/$safeStation/$safeOperator"
        }
    }
}
```

Тека com/example/npp_radiation_monitor/ui/components/form
Файл EmailField.kt

```
package com.example.npp_radiation_monitor.ui.components.form

import androidx.compose.foundation.layout.fillMaxWidth
import androidx.compose.foundation.text.KeyboardOptions
import androidx.compose.material.icons.Icons
import androidx.compose.material.icons.filled.Email
import androidx.compose.material3.Icon
import androidx.compose.material3.OutlinedTextField
import androidx.compose.material3.Text
import androidx.compose.runtime.Composable
import androidx.compose.ui.Modifier
import androidx.compose.ui.text.input.KeyboardType
```

```
/**
 * Текстове поле введення email оператора (EmailField).
 */
@Composable
fun EmailField(
    value: String,
    onValueChange: (String) -> Unit,
    error: String?
) {
    val isError = error != null

    OutlinedTextField(
        value = value,
        onValueChange = onValueChange,
        label = { Text("Електронна пошта оператора") },
        leadingIcon = { Icon(Icons.Default.Email, contentDescription =
null) },
        keyboardOptions = KeyboardOptions(keyboardType =
KeyboardType.Email),
        isError = isError,
        supportingText = {
            if (isError) {
                Text(error)
            } else {
                Text("Формат: operator@npp.gov.ua")
            }
        },
        singleLine = true,
        modifier = Modifier.fillMaxWidth()
    )
}
```

Тека com/example/npp_radiation_monitor/ui/components/form

Файл FormHeader.kt

```
package com.example.npp_radiation_monitor.ui.components.form

import androidx.compose.foundation.layout.Column
import androidx.compose.foundation.layout.Spacer
import androidx.compose.foundation.layout.fillMaxWidth
import androidx.compose.foundation.layout.height
import androidx.compose.material.icons.Icons
import androidx.compose.material.icons.filled.Build
import androidx.compose.material3.Icon
import androidx.compose.material3.MaterialTheme
import androidx.compose.material3.Text
import androidx.compose.runtime.Composable
import androidx.compose.ui.Alignment
import androidx.compose.ui.Modifier
```

```
import androidx.compose.ui.text.font.FontWeight
import androidx.compose.ui.unit.dp

/**
 * Заголовок форми налаштування станції (FormHeader).
 * Відображає іконку, назву форми та пояснювальний підзаголовок.
 */
@Composable
fun FormHeader() {
    Column(
        modifier = Modifier.fillMaxWidth(),
        horizontalAlignment = Alignment.CenterHorizontally
    ) {
        Icon(
            imageVector = Icons.Default.Build,
            contentDescription = null,
            tint = MaterialTheme.colorScheme.primary,
            modifier = Modifier.height(48.dp)
        )
        Spacer(modifier = Modifier.height(8.dp))
        Text(
            text = "Реєстрація станції АЕС",
            style = MaterialTheme.typography.headlineSmall,
            fontWeight = FontWeight.Bold
        )
        Text(
            text = "Введіть параметри обладнання та дані оператора",
            style = MaterialTheme.typography.bodyMedium,
            color = MaterialTheme.colorScheme.onSurfaceVariant
        )
    }
}
```

Тека com/example/npp_radiation_monitor/ui/components/form

Файл PasswordField.kt

```
package com.example.npp_radiation_monitor.ui.components.form

import androidx.compose.foundation.layout.fillMaxWidth
import androidx.compose.foundation.text.KeyboardOptions
import androidx.compose.material.icons.Icons
import androidx.compose.material.icons.filled.Lock
import androidx.compose.material.icons.filled.Visibility
import androidx.compose.material.icons.filled.VisibilityOff
import androidx.compose.material3.Icon
import androidx.compose.material3.IconButton
import androidx.compose.material3.OutlinedTextField
import androidx.compose.material3.Text
import androidx.compose.runtime.Composable
```

```
import androidx.compose.runtime.getValue
import androidx.compose.runtime.mutableStateOf
import androidx.compose.runtime.remember
import androidx.compose.runtime.setValue
import androidx.compose.ui.Modifier
import androidx.compose.ui.text.input.KeyboardType
import androidx.compose.ui.text.input.PasswordVisualTransformation
import androidx.compose.ui.text.input.VisualTransformation

/**
 * Текстове поле введення пароля (PasswordField) із можливістю
приховування символів.
 */
@Composable
fun PasswordField(
    value: String,
    onValueChange: (String) -> Unit,
    error: String?
) {
    var passwordVisible by remember { mutableStateOf(false) }
    val isError = error != null

    OutlinedTextField(
        value = value,
        onValueChange = onValueChange,
        label = { Text("Пароль доступу") },
        leadingIcon = { Icon(Icons.Default.Lock, contentDescription =
null) },
        trailingIcon = {
            IconButton(onClick = { passwordVisible = !passwordVisible })
{
                Icon(
                    imageVector = if (passwordVisible)
Icons.Default.Visibility else Icons.Default.VisibilityOff,
                    contentDescription = if (passwordVisible) "Сховати
пароль" else "Показати пароль"
                )
            }
        },
        visualTransformation = if (passwordVisible)
VisualTransformation.None else PasswordVisualTransformation(),
        keyboardOptions = KeyboardOptions(keyboardType =
KeyboardType.Password),
        isError = isError,
        supportingText = {
            if (isError) {
                Text(error)
            } else {
                Text("Мінімум 6 символів")
            }
        },
```

```
        singleLine = true,
        modifier = Modifier.fillMaxWidth()
    )
}
```

Тека com/example/npp_radiation_monitor/ui/components/form
Файл PowerField.kt

```
package com.example.npp_radiation_monitor.ui.components.form

import androidx.compose.foundation.layout.fillMaxWidth
import androidx.compose.foundation.text.KeyboardOptions
import androidx.compose.material.icons.Icons
import androidx.compose.material.icons.filled.Info
import androidx.compose.material3.Icon
import androidx.compose.material3.OutlinedTextField
import androidx.compose.material3.Text
import androidx.compose.runtime.Composable
import androidx.compose.ui.Modifier
import androidx.compose.ui.text.input.KeyboardType

/**
 * Текстове поле введення встановленої потужності станції (PowerField).
 */
@Composable
fun PowerField(
    value: String,
    onValueChange: (String) -> Unit,
    error: String?
) {
    val isError = error != null

    OutlinedTextField(
        value = value,
        onValueChange = onValueChange,
        label = { Text("Встановлена потужність (МВт)") },
        leadingIcon = { Icon(Icons.Default.Info, contentDescription =
null) },
        keyboardOptions = KeyboardOptions(keyboardType =
KeyboardType.Number),
        isError = isError,
        supportingText = {
            if (isError) {
                Text(error)
            } else {
                Text("Значення повинно бути додатним числом")
            }
        },
        singleLine = true,
```

```
        modifier = Modifier.fillMaxWidth()
    )
}
```

Тека com/example/npp_radiation_monitor/ui/components/form
Файл StationNameField.kt

```
package com.example.npp_radiation_monitor.ui.components.form

import androidx.compose.foundation.layout.fillMaxWidth
import androidx.compose.material.icons.Icons
import androidx.compose.material.icons.filled.Home
import androidx.compose.material3.Icon
import androidx.compose.material3.OutlinedTextField
import androidx.compose.material3.Text
import androidx.compose.runtime.Composable
import androidx.compose.ui.Modifier

/**
 * Текстове поле введення назви станції/блоку АЕС (StationNameField).
*/
@Composable
fun StationNameField(
    value: String,
    onValueChange: (String) -> Unit,
    error: String?
) {
    val isError = error != null

    OutlinedTextField(
        value = value,
        onValueChange = onValueChange,
        label = { Text("Назва станції / Блоку") },
        leadingIcon = { Icon(Icons.Default.Home, contentDescription =
null) },
        isError = isError,
        supportingText = {
            if (isError) {
                Text(error)
            } else {
                Text("Наприклад: Запорізька АЕС, Блок №1")
            }
        },
        singleLine = true,
        modifier = Modifier.fillMaxWidth()
    )
}
```

Тека com/example/npp_radiation_monitor/ui/components/form
Файл SubmitButton.kt

```
package com.example.npp_radiation_monitor.ui.components.form

import androidx.compose.foundation.layout.fillMaxWidth
import androidx.compose.foundation.layout.height
import androidx.compose.material3.Button
import androidx.compose.material3.CircularProgressIndicator
import androidx.compose.material3.MaterialTheme
import androidx.compose.material3.Text
import androidx.compose.runtime.Composable
import androidx.compose.ui.Modifier
import androidx.compose.ui.unit.dp
import androidx.compose.ui.unit.sp

/**
 * Кнопка відправки форми (SubmitButton) із підтримкою стану завантаження
(Loading State).
 */
@Composable
fun SubmitButton(
    onClick: () -> Unit,
    isLoading: Boolean,
    enabled: Boolean = true
) {
    Button(
        onClick = onClick,
        enabled = enabled && !isLoading,
        modifier = Modifier
            .fillMaxWidth()
            .height(50.dp)
    ) {
        if (isLoading) {
            CircularProgressIndicator(
                color = MaterialTheme.colorScheme.onPrimary,
                strokeWidth = 3.dp
            )
        } else {
            Text(text = "Зберегти налаштування", fontSize = 16.sp)
        }
    }
}
```

Тека com/example/npp_radiation_monitor/ui/components
Файл AuthDialog.kt

```
package com.example.npp_radiation_monitor.ui.components

import androidx.compose.foundation.layout.Column
import androidx.compose.foundation.layout.Spacer
import androidx.compose.foundation.layout.fillMaxWidth
import androidx.compose.foundation.layout.height
import androidx.compose.foundation.text.KeyboardOptions
import androidx.compose.material.icons.Icons
import androidx.compose.material.icons.filled.Email
import androidx.compose.material.icons.filled.Lock
import androidx.compose.material.icons.filled.Person
import androidx.compose.material.icons.filled.Visibility
import androidx.compose.material.icons.filled.VisibilityOff
import androidx.compose.material3.AlertDialog
import androidx.compose.material3.Button
import androidx.compose.material3.Icon
import androidx.compose.material3.IconButton
import androidx.compose.material3.MaterialTheme
import androidx.compose.material3.OutlinedTextField
import androidx.compose.material3.Text
import androidx.compose.material3.TextButton
import androidx.compose.runtime.Composable
import androidx.compose.runtime.getValue
import androidx.compose.runtime.mutableStateOf
import androidx.compose.runtime.remember
import androidx.compose.runtime.setValue
import androidx.compose.ui.Modifier
import androidx.compose.ui.graphics.Color
import androidx.compose.ui.text.font.FontWeight
import androidx.compose.ui.text.input.KeyboardType
import androidx.compose.ui.text.input.PasswordVisualTransformation
import androidx.compose.ui.text.input.VisualTransformation
import androidx.compose.ui.unit.dp

fun checkErrors(email: String, password: String): String? {
 return when {
 email.isBlank() -> "Заповніть електронну пошту"
 password.length < 6 -> "Пароль повинен містити більше 6 символів"
 else -> null
 }
}

@Composable
fun Errors(error: String?) {
 if (error == null) return
 if (error.isNotEmpty()) {
 Text(
 text = error,
```

```
 color = Color.Red,
 fontWeight = FontWeight.Bold
 )
 }
}

@Composable
fun AuthDialog(
 onDismissRequest: () -> Unit,
    onSuccess: (operatorName: String) -> Unit
) {
    // Перемикач режимів: false = вхід, true = реєстрація
    var isRegisterMode by remember { mutableStateOf(false) }

    // Поля введення
    var email by remember { mutableStateOf("") }
    var password by remember { mutableStateOf("") }
    var name by remember { mutableStateOf("") }
    var passwordVisible by remember { mutableStateOf(false) }

    // Булева змінна, яка каже, чи показувати помилку, чи ні
    var showError by remember { mutableStateOf(false) }

    AlertDialog(
        onDismissRequest = onDismissRequest,
        title = {
            Text(
                text = if (isRegisterMode) "Реєстрація оператора АЕС"
else "Вхід у систему АЕС",
                style = MaterialTheme.typography.titleLarge,
                fontWeight = FontWeight.Bold
            )
        },
        text = {
            Column(modifier = Modifier.fillMaxWidth()) {
                // Поле імені (показується тільки при реєстрації)
                if (isRegisterMode) {
                    OutlinedTextField(
                        value = name,
                        onValueChange = { name = it; showError = false },
                        label = { Text("ПІБ Оператора") },
                        leadingIcon = { Icon(Icons.Default.Person,
contentDescription = null) },
                        singleLine = true,
                        modifier = Modifier.fillMaxWidth()
                    )
                    Spacer(modifier = Modifier.height(8.dp))
                }

                // Поле email
                OutlinedTextField(
```

```
                    value = email,
                    onValueChange = { email = it; showError = false },
                    label = { Text("Електронна пошта") },
                    leadingIcon = { Icon(Icons.Default.Email,
contentDescription = null) },
                    keyboardOptions = KeyboardOptions(keyboardType =
KeyboardType.Email),
                    singleLine = true,
                    modifier = Modifier.fillMaxWidth()
                )

                Spacer(modifier = Modifier.height(8.dp))

                // Поле password
                OutlinedTextField(
                    value = password,
                    onValueChange = { password = it; showError = false },
                    label = { Text("Пароль") },
                    leadingIcon = { Icon(Icons.Default.Lock,
contentDescription = null) },
                    trailingIcon = {
                        IconButton(onClick = { passwordVisible = !
passwordVisible }) {
                            Icon(
                                imageVector = if (passwordVisible)
Icons.Default.Visibility else Icons.Default.VisibilityOff,
                                contentDescription = null
                            )
                        }
                    },
                    visualTransformation = if (passwordVisible)
VisualTransformation.None else PasswordVisualTransformation(),
                    keyboardOptions = KeyboardOptions(keyboardType =
KeyboardType.Password),
                    singleLine = true,
                    modifier = Modifier.fillMaxWidth()
                )

                Spacer(modifier = Modifier.height(8.dp))

                if (showError) Errors(checkErrors(email, password))

                Spacer(modifier=Modifier.height(8.dp))

                // Кнопка перемикання вхід / реєстрація
                TextButton(
                    onClick = { isRegisterMode = !isRegisterMode;
showError = false },
                    modifier = Modifier.fillMaxWidth()
                ) {
                    Text(
```

```
                        text = if (isRegisterMode) "Вже є акаунт? Увійти"
else "Немає акаунту? Зареєструватися"
                    )
                }
            }
        },
        confirmButton = {
            Button(
                onClick = {
                    // Перевіряємо, чи є помилки
                    if (checkErrors(email, password) != null) {
                        showError = true
                    } else {
                        onSuccess(email)
                    }
                }
            ) {
                Text(if (isRegisterMode) "Зареєструватися" else "Увійти")
            }
        },
        dismissButton = {
            TextButton(onClick = onDismissRequest) {
                Text("Скасувати")
            }
        }
    )
}
```

Тека com/example/npp_radiation_monitor/ui/components
Файл AuthHeader.kt

```
package com.example.npp_radiation_monitor.ui.components

import androidx.compose.foundation.layout.Arrangement
import androidx.compose.foundation.layout.Row
import androidx.compose.foundation.layout.Spacer
import androidx.compose.foundation.layout.fillMaxWidth
import androidx.compose.foundation.layout.width
import androidx.compose.material.icons.Icons
import androidx.compose.material.icons.automirrored.filled.ExitToApp
import androidx.compose.material.icons.filled.AccountCircle
import androidx.compose.material3.Icon
import androidx.compose.material3.MaterialTheme
import androidx.compose.material3.OutlinedButton
import androidx.compose.material3.Text
import androidx.compose.material3.TextButton
import androidx.compose.runtime.Composable
import androidx.compose.ui.Alignment
import androidx.compose.ui.Modifier
```

```
import androidx.compose.ui.text.font.FontWeight
import androidx.compose.ui.unit.dp

@Composable
fun AuthHeader(operatorEmail: String?, onAuthClick: () -> Unit,
onLogoutClick: () -> Unit) {
    Row(
            modifier = Modifier.fillMaxWidth(),
            horizontalArrangement = Arrangement.SpaceBetween,
            verticalAlignment = Alignment.CenterVertically
        ) {
            Row(verticalAlignment = Alignment.CenterVertically) {
                Icon(
                    imageVector = Icons.Default.AccountCircle,
                    contentDescription = "Профіль",
                    tint = MaterialTheme.colorScheme.primary
                )
                Spacer(modifier = Modifier.width(8.dp))
                Text(
                    text = if (operatorEmail != null) "Оператор:
$operatorEmail" else "Статус: Гість",
                    style = MaterialTheme.typography.bodyMedium,
                    fontWeight = FontWeight.SemiBold
                )
            }

            if (operatorEmail == null) {
                OutlinedButton(onClick = onAuthClick) {
                    Text("Увійти")
                }
            } else {
                TextButton(onClick = onLogoutClick) {
                    Icon(Icons.AutoMirrored.Filled.ExitToApp,
contentDescription = null)
                    Spacer(modifier = Modifier.width(4.dp))
                    Text("Вийти")
                }
            }
        }
}
```

Тека com/example/npp_radiation_monitor/ui/components

Файл HistoryItem.kt

```
package com.example.npp_radiation_monitor.ui.components

import androidx.compose.foundation.layout.Arrangement
import androidx.compose.foundation.layout.Column
import androidx.compose.foundation.layout.Row
import androidx.compose.foundation.layout.fillMaxWidth
```

```
import androidx.compose.foundation.layout.padding
import androidx.compose.material3.Card
import androidx.compose.material3.CardDefaults
import androidx.compose.material3.MaterialTheme
import androidx.compose.material3.Text
import androidx.compose.runtime.Composable
import androidx.compose.ui.Alignment
import androidx.compose.ui.Modifier
import androidx.compose.ui.text.font.FontWeight
import androidx.compose.ui.unit.dp
import com.example.npp_radiation_monitor.data.model.RadiationReading
import java.util.Locale

/**
 * Окремий елемент списку історії вимірювань (HistoryItem).
 * Відображає номер вимірювання, час проведення та показник радіації з
кольором статусу.
 */
@Composable
fun HistoryItem(reading: RadiationReading) {
    Card(
        modifier = Modifier.fillMaxWidth(),
        colors = CardDefaults.cardColors(containerColor =
MaterialTheme.colorScheme.surfaceVariant)
    ) {
        Row(
            modifier = Modifier
                .padding(16.dp)
                .fillMaxWidth(),
            horizontalArrangement = Arrangement.SpaceBetween,
            verticalAlignment = Alignment.CenterVertically
        ) {
            Column {
                Text(text = "Замір #${reading.id}", style =
MaterialTheme.typography.labelMedium)
                Text(text = reading.timestamp, style =
MaterialTheme.typography.bodySmall)
            }
            Text(
                text = String.format(locale = Locale.UK, "%.2f мкЗв /
год", reading.value),
                style = MaterialTheme.typography.titleMedium,
                fontWeight = FontWeight.Bold,
                color = reading.status.color
            )
        }
    }
}
```

Тека com/example/npp_radiation_monitor/ui/components
Файл StatusCard.kt

```
package com.example.npp_radiation_monitor.ui.components

import androidx.compose.animation.animateColorAsState
import androidx.compose.animation.core.animateFloatAsState
import androidx.compose.foundation.layout.Arrangement
import androidx.compose.foundation.layout.Column
import androidx.compose.foundation.layout.fillMaxSize
import androidx.compose.foundation.layout.fillMaxWidth
import androidx.compose.foundation.layout.height
import androidx.compose.material3.Card
import androidx.compose.material3.CardDefaults
import androidx.compose.material3.MaterialTheme
import androidx.compose.material3.Text
import androidx.compose.runtime.Composable
import androidx.compose.runtime.getValue
import androidx.compose.ui.Alignment
import androidx.compose.ui.Modifier
import androidx.compose.ui.graphics.Color
import androidx.compose.ui.text.font.FontWeight
import androidx.compose.ui.unit.dp
import androidx.compose.ui.unit.sp
import com.example.npp_radiation_monitor.data.model.RadiationReading
import java.util.Locale

/**
 * Компонент картки статусу радіаційного фону (StatusCard).
 * Відображає поточний рівень радіації із анімованим зміненням кольору
 * та значення залежно від статусу безпеки.
 */
@Composable
fun StatusCard(
    reading: RadiationReading,
    modifier: Modifier = Modifier
) {
    // Анімації значення радіації та кольору картки
    val animatedRadiation by animateFloatAsState(targetValue =
reading.value)
    val animatedColor by animateColorAsState(targetValue =
reading.status.color)

    Card(
        modifier = modifier
            .fillMaxWidth()
            .height(180.dp),
        colors = CardDefaults.cardColors(containerColor = animatedColor),
        elevation = CardDefaults.cardElevation(defaultElevation = 8.dp)
    ) {
        Column(
```

```
            modifier = Modifier.fillMaxSize(),
            verticalArrangement = Arrangement.Center,
            horizontalAlignment = Alignment.CenterHorizontally
        ) {
            // Текст статусу (Норма, Увага, Небезпека)
            Text(
                text = "Статус: ${reading.status.title}",
                color = Color.White,
                style = MaterialTheme.typography.titleMedium
            )
            // Анімоване числове значення радіації
            Text(
                text = String.format(locale = Locale.UK, "%.2f",
animatedRadiation),
                color = Color.White,
                fontSize = 56.sp,
                fontWeight = FontWeight.Black
            )
            // Одиниці вимірювання
            Text(
                text = "мкЗв/год",
                color = Color.White,
                style = MaterialTheme.typography.bodyLarge
            )
        }
    }
}
```

Тека com/example/npp_radiation_monitor/ui/screens

Файл HistoryScreen.kt

```
package com.example.npp_radiation_monitor.ui.screens

import androidx.compose.foundation.layout.Arrangement
import androidx.compose.foundation.layout.Column
import androidx.compose.foundation.layout.Row
import androidx.compose.foundation.layout.Spacer
import androidx.compose.foundation.layout.fillMaxWidth
import androidx.compose.foundation.layout.padding
import androidx.compose.foundation.lazy.LazyColumn
import androidx.compose.foundation.lazy.items
import androidx.compose.material3.Button
import androidx.compose.material3.HorizontalDivider
import androidx.compose.material3.MaterialTheme
import androidx.compose.material3.Text
import androidx.compose.runtime.Composable
import androidx.compose.ui.Alignment
import androidx.compose.ui.Modifier
import androidx.compose.ui.text.font.FontWeight
import androidx.compose.ui.unit.dp
```

```
import androidx.compose.ui.unit.sp
import androidx.navigation.NavController
import com.example.npp_radiation_monitor.data.model.RadiationReading
import com.example.npp_radiation_monitor.navigation.Screen
import com.example.npp_radiation_monitor.ui.components.HistoryItem

/**
 * Екран історії вимірювань (HistoryScreen).
 * Відображає список усіх збережених вимірювань радіаційного фону.
 */
@Composable
fun HistoryScreen(
    navController: NavController,
    history: List<RadiationReading>
) {
    Column(
        modifier = Modifier
            .fillMaxWidth()
            .padding(16.dp)
            .padding(top = 40.dp),
        horizontalAlignment = Alignment.CenterHorizontally
    ) {
        // Кнопка повернення на головний екран
        Button(
            onClick =
{ navController.navigate(Screen.MainScreen.route) },
            modifier = Modifier.fillMaxWidth()
        ) {
            Text(text = "Повернутися на головну", fontSize = 16.sp)
        }

        Spacer(modifier = Modifier.padding(8.dp))

        // Кнопка повернення на екран моніторингу
        Button(
            onClick = { navController.popBackStack() },
            modifier = Modifier.fillMaxWidth()
        ) {
            Text(text = "Повернутися до моніторингу радіації", fontSize =
16.sp)
        }

        Spacer(modifier = Modifier.padding(8.dp))

        // Заголовок екрана історії
        Row(
            modifier = Modifier.fillMaxWidth(),
            horizontalArrangement = Arrangement.Start
        ) {
            Text(
                text = "Історія вимірювань",
```

```
                style = MaterialTheme.typography.titleLarge,
                fontWeight = FontWeight.SemiBold
            )
        }

        HorizontalDivider(modifier = Modifier.padding(vertical = 8.dp))

        // Список усіх вимірювань
        LazyColumn(
            modifier = Modifier
                .fillMaxWidth()
                .padding(bottom = 16.dp),
            verticalArrangement = Arrangement.spacedBy(8.dp)
        ) {
            items(history) { item ->
                HistoryItem(reading = item)
            }
        }
    }
}
```

Тека com/example/npp_radiation_monitor/ui/screens

Файл MainScreen.kt

```
package com.example.npp_radiation_monitor.ui.screens

import androidx.compose.foundation.layout.Arrangement
import androidx.compose.foundation.layout.Column
import androidx.compose.foundation.layout.Spacer
import androidx.compose.foundation.layout.fillMaxHeight
import androidx.compose.foundation.layout.fillMaxWidth
import androidx.compose.foundation.layout.height
import androidx.compose.foundation.layout.padding
import androidx.compose.material3.Button
import androidx.compose.material3.OutlinedTextField
import androidx.compose.material3.Text
import androidx.compose.runtime.Composable
import androidx.compose.runtime.getValue
import androidx.compose.runtime.mutableStateOf
import androidx.compose.runtime.remember
import androidx.compose.runtime.setValue
import androidx.compose.ui.Alignment
import androidx.compose.ui.Modifier
import androidx.compose.ui.text.font.FontWeight
import androidx.compose.ui.unit.dp
import androidx.compose.ui.unit.sp
import androidx.navigation.NavController
import com.example.npp_radiation_monitor.navigation.Screen

/**
```

```
 * Головний екран застосунку (MainScreen).
 * Містить форму введення імені оператора та кнопку для переходу
 * до моніторингу радіації з передачею даних через аргументи навігації.
 */
@Composable
fun MainScreen(navController: NavController) {
    // Стан для збереження введеного імені оператора
    var operatorName by remember { mutableStateOf("") }

    Column(
        modifier = Modifier
            .fillMaxWidth()
            .fillMaxHeight()
            .padding(16.dp),
        horizontalAlignment = Alignment.CenterHorizontally,
        verticalArrangement = Arrangement.Center
    ) {
        // Заголовок екрана
        Text(
            text = "Головний екран",
            fontSize = 24.sp,
            fontWeight = FontWeight.Bold
        )

        Spacer(modifier = Modifier.padding(16.dp))

        // Форма введення імені оператора
        OutlinedTextField(
            value = operatorName,
            onValueChange = { operatorName = it },
            label = { Text("Введіть ім'я оператора") },
            modifier = Modifier.fillMaxWidth(),
            singleLine = true
        )

        Spacer(modifier = Modifier.height(16.dp))

        // Кнопка переходу до екрана моніторингу
        Button(
            modifier = Modifier.fillMaxWidth(),
            onClick = {
                navController.navigate(
                    Screen.RadiationScreen.createRoute(operatorName =
operatorName)
                )
            }
        ) {
            Text(text = "Моніторинг радіації", fontSize = 16.sp)
        }

        Spacer(modifier = Modifier.height(12.dp))
```

```
        // Кнопка переходу до повної форми реєстрації/налаштування
станції (Заняття №6)
        Button(
            modifier = Modifier.fillMaxWidth(),
            onClick = {
                navController.navigate(Screen.FormScreen.route)
            }
        ) {
            Text(text = "Форма реєстрації станції", fontSize = 16.sp)
        }
    }
}
```

Тека com/example/npp_radiation_monitor/ui/screens

Файл RadiationScreen.kt

```
package com.example.npp_radiation_monitor.ui.screens

import androidx.compose.foundation.layout.Column
import androidx.compose.foundation.layout.Row
import androidx.compose.foundation.layout.Spacer
import androidx.compose.foundation.layout.fillMaxSize
import androidx.compose.foundation.layout.fillMaxWidth
import androidx.compose.foundation.layout.height
import androidx.compose.foundation.layout.padding
import androidx.compose.foundation.layout.width
import androidx.compose.material.icons.Icons
import androidx.compose.material.icons.filled.AccountCircle
import androidx.compose.material.icons.filled.Warning
import androidx.compose.material3.Button
import androidx.compose.material3.HorizontalDivider
import androidx.compose.material3.Icon
import androidx.compose.material3.MaterialTheme
import androidx.compose.material3.Text
import androidx.compose.runtime.Composable
import androidx.compose.runtime.getValue
import androidx.compose.runtime.mutableStateOf
import androidx.compose.runtime.remember
import androidx.compose.runtime.setValue
import androidx.compose.runtime.snapshots.SnapshotStateList
import androidx.compose.ui.Alignment
import androidx.compose.ui.Modifier
import androidx.compose.ui.text.font.FontWeight
import androidx.compose.ui.unit.dp
import androidx.compose.ui.unit.sp
import androidx.navigation.NavController
import com.example.npp_radiation_monitor.data.RadiationManager
import com.example.npp_radiation_monitor.data.model.RadiationReading
```

```
import com.example.npp_radiation_monitor.navigation.Screen
import com.example.npp_radiation_monitor.ui.components.StatusCard

/**
 * Екран моніторингу радіації (RadiationScreen).
 * Відображає поточні показники радіаційного фону, дозволяє генерувати
 * нові вимірювання та зберігає їх у спільну історію.
 */
@Composable
fun RadiationScreen(
    navController: NavController,
    radiationManager: RadiationManager,
    history: SnapshotStateList<RadiationReading>,
    operatorName: String = "Черговий оператор",
    stationName: String = "Моніторинг АЕС"
) {
    // Поточне вимірювання датчика
    var currentReading by remember
{ mutableStateOf(radiationManager.createInitialReading()) }

    Column(
        modifier = Modifier
            .fillMaxSize()
            .padding(16.dp)
            .padding(top = 40.dp),
        horizontalAlignment = Alignment.CenterHorizontally,
    ) {
        // Рядок із інформацією про оператора
        Row(
            modifier = Modifier
                .fillMaxWidth()
                .padding(bottom = 8.dp),
            verticalAlignment = Alignment.CenterVertically
        ) {
            Icon(Icons.Default.AccountCircle, contentDescription = null)
            Spacer(modifier = Modifier.width(8.dp))
            Text(
                text = "Оператор: $operatorName",
                style = MaterialTheme.typography.titleMedium,
                fontWeight = FontWeight.SemiBold
            )
        }

        // Кнопка повернення на головну
        Button(
            onClick =
{ navController.navigate(Screen.MainScreen.route) },
            modifier = Modifier.fillMaxWidth()
        ) {
            Text(text = "Перейти на головну", fontSize = 16.sp)
        }
```

```
        HorizontalDivider(modifier = Modifier.padding(vertical = 12.dp))

        // Заголовок станції (передане значення)
        Text(
            text = stationName,
            style = MaterialTheme.typography.headlineMedium,
            fontWeight = FontWeight.Bold,
            modifier = Modifier.padding(bottom = 12.dp)
        )

        // Картка поточного стану радіації
        StatusCard(reading = currentReading)

        Spacer(modifier = Modifier.height(12.dp))

        // Кнопка оновлення показників датчика
        Button(
            onClick = {
                val newReading = radiationManager.generateNextReading()
                currentReading = newReading
                history.add(0, newReading)
            },
            modifier = Modifier.fillMaxWidth(),
            shape = MaterialTheme.shapes.large
        ) {
            Icon(Icons.Default.Warning, contentDescription = null)
            Spacer(Modifier.width(8.dp))
            Text("Оновити показники датчика")
        }

        Spacer(modifier = Modifier.height(16.dp))

        // Кнопка переходу до екрана історії
        Button(
            onClick =
{ navController.navigate(Screen.HistoryScreen.route) },
            modifier = Modifier.fillMaxWidth()
        ) {
            Text(text = "Переглянути історію", fontSize = 16.sp)
        }
    }
}
```

Тека com/example/npp_radiation_monitor/ui/screens

Файл StationFormScreen.kt

```
package com.example.npp_radiation_monitor.ui.screens

import androidx.compose.foundation.layout.Arrangement
```

```
import androidx.compose.foundation.layout.Column
import androidx.compose.foundation.layout.Spacer
import androidx.compose.foundation.layout.fillMaxSize
import androidx.compose.foundation.layout.fillMaxWidth
import androidx.compose.foundation.layout.height
import androidx.compose.foundation.layout.padding
import androidx.compose.foundation.rememberScrollState
import androidx.compose.foundation.verticalScroll
import androidx.compose.material3.AlertDialog
import androidx.compose.material3.Button
import androidx.compose.material3.HorizontalDivider
import androidx.compose.material3.MaterialTheme
import androidx.compose.material3.Text
import androidx.compose.material3.TextButton
import androidx.compose.runtime.Composable
import androidx.compose.runtime.LaunchedEffect
import androidx.compose.runtime.getValue
import androidx.compose.runtime.mutableStateOf
import androidx.compose.runtime.remember
import androidx.compose.runtime.setValue
import androidx.compose.ui.Alignment
import androidx.compose.ui.Modifier
import androidx.compose.ui.unit.dp
import androidx.navigation.NavController
import com.example.npp_radiation_monitor.navigation.Screen
import com.example.npp_radiation_monitor.ui.components.form.EmailField
import com.example.npp_radiation_monitor.ui.components.form.FormHeader
import com.example.npp_radiation_monitor.ui.components.form.PasswordField
import com.example.npp_radiation_monitor.ui.components.form.PowerField
import
com.example.npp_radiation_monitor.ui.components.form.StationNameField
import com.example.npp_radiation_monitor.ui.components.form.SubmitButton
import kotlinx.coroutines.delay
import kotlin.time.Duration.Companion.milliseconds

/**
 * Екран реєстрації та налаштування станції (StationFormScreen).
 * Реалізує комплексну форма введення даних із перевіркою (валідацією),
 * підказками користувачу (Helper Text) та індикатором завантаження
(Loading State).
 */
@Composable
fun StationFormScreen(navController: NavController) {
    // Поля введення
    var stationName by remember { mutableStateOf("") }
    var email by remember { mutableStateOf("") }
    var password by remember { mutableStateOf("") }
    var power by remember { mutableStateOf("") }

    // Повідомлення про помилки
    var stationNameError by remember { mutableStateOf<String?>(null) }
```

```
    var emailError by remember { mutableStateOf<String?>(null) }
    var passwordError by remember { mutableStateOf<String?>(null) }
    var powerError by remember { mutableStateOf<String?>(null) }

    // Стан завантаження та діалог успіху
    var isLoading by remember { mutableStateOf(false) }
    var showSuccessDialog by remember { mutableStateOf(false) }

    // Симуляція мережевого запиту / завантаження при відправці теми
    LaunchedEffect(isLoading) {
        if (isLoading) {
            delay(1500.milliseconds)
            isLoading = false
            showSuccessDialog = true
        }
    }

    // Функція валідації всієї форми
    fun validateForm(): Boolean {
        var isValid = true

        // 1. Валідація назви станції
        if (stationName.isBlank()) {
            stationNameError = "Заповніть назву станції"
            isValid = false
        } else {
            stationNameError = null
        }

        // 2. Валідація Email
        val emailPattern = Regex("^[A-Za-z0-9+_.-]+@(.+)$")
        if (email.isBlank()) {
            emailError = "Заповніть електронну пошту"
            isValid = false
        } else if (!email.matches(emailPattern)) {
            emailError = "Некоректний email (наприклад:
operator@npp.gov.ua)"
            isValid = false
        } else {
            emailError = null
        }

        // 3. Валідація пароля
        if (password.length < 6) {
            passwordError = "Пароль повинен містити не менше 6 символів"
            isValid = false
        } else {
            passwordError = null
        }

        // 4. Валідація потужності
```

```
    val powerValue = power.toFloatOrNull()
    if (power.isBlank()) {
        powerError = "Заповніть значення потужності"
        isValid = false
    } else if (powerValue == null || powerValue <= 0) {
        powerError = "Вкажіть число більше 0 МВт"
        isValid = false
    } else {
        powerError = null
    }

    return isValid
}

Column(
    modifier = Modifier
        .fillMaxSize()
        .verticalScroll(rememberScrollState())
        .padding(16.dp)
        .padding(top = 40.dp),
    horizontalAlignment = Alignment.CenterHorizontally,
    verticalArrangement = Arrangement.spacedBy(12.dp)
) {
    // Заголовок форми
    FormHeader()

    HorizontalDivider(modifier = Modifier.padding(vertical = 8.dp))

    // Поле назви станції
    StationNameField(
        value = stationName,
        onValueChange = {
            stationName = it
            if (stationNameError != null) stationNameError = null
        },
        error = stationNameError
    )

    // Поле email
    EmailField(
        value = email,
        onValueChange = {
            email = it
            if (emailError != null) emailError = null
        },
        error = emailError
    )

    // Поле пароля
    PasswordField(
        value = password,
```

```
            onValueChange = {
                password = it
                if (passwordError != null) passwordError = null
            },
            error = passwordError
        )

        // Поле потужності
        PowerField(
            value = power,
            onValueChange = {
                power = it
                if (powerError != null) powerError = null
            },
            error = powerError
        )

        Spacer(modifier = Modifier.height(16.dp))

        // Кнопка відправки форми
        SubmitButton(
            onClick = {
                if (validateForm()) {
                    isLoading = true
                }
            },
            isLoading = isLoading
        )

        // Кнопка скасування / повернення
        TextButton(
            onClick = { navController.popBackStack() },
            modifier = Modifier.fillMaxWidth()
        ) {
            Text("Скасувати та повернутися")
        }
    }

    // Діалог успішної реєстрації/налаштування
    if (showSuccessDialog) {
        AlertDialog(
            onDismissRequest = { showSuccessDialog = false },
            title = { Text("Успішне збереження", style =
MaterialTheme.typography.titleLarge) },
            text = { Text("Параметри станції «$stationName» та оператора
«$email» успішно збережено!") },
            confirmButton = {
                Button(
                    onClick = {
                        showSuccessDialog = false
                        navController.navigate(
```

```
                                Screen.RadiationScreen.createRoute(
                                    stationName = stationName,
                                    operatorName = email
                                )
                            )
                        }
                    ) {
                        Text("Перейти до моніторингу")
                    }
                }
            )
        }
    }
```

ПРОГРАМНИЙ КОД ПРАКТИЧНОГО ЗАНЯТТЯ №7

Тека com/example/npp_radiation_monitor

Файл MainActivity.kt

```
package com.example.npp_radiation_monitor

import android.os.Bundle
import androidx.activity.ComponentActivity
import androidx.activity.compose.setContent
import androidx.activity.enableEdgeToEdge
import com.example.npp_radiation_monitor.navigation.Navigation
import
com.example.npp_radiation_monitor.ui.theme.NPP_Radiation_MonitorTheme

class MainActivity : ComponentActivity() {
    override fun onCreate(savedInstanceState: Bundle?) {
        super.onCreate(savedInstanceState)
        enableEdgeToEdge()
        setContent {
            NPP_Radiation_MonitorTheme {
                Navigation()
            }
        }
    }
}
```

Тека com/example/npp_radiation_monitor/data/model

Файл RadiationReading.kt

```
package com.example.npp_radiation_monitor.data.model

data class RadiationReading(
    val id: Int,
    val value: Float,
    val timestamp: String,
    val status: RadiationStatus = RadiationStatus.fromValue(value)
)
```

Тека com/example/npp_radiation_monitor/data

Файл RadiationManager.kt

```
package com.example.npp_radiation_monitor.data

import com.example.npp_radiation_monitor.data.model.RadiationReading
import com.example.npp_radiation_monitor.data.sensor.RadiationSensor
import
com.example.npp_radiation_monitor.data.sensor.SimulatedRadiationSensor
```

```
class RadiationManager(
    private val sensor: RadiationSensor = SimulatedRadiationSensor()
) {
    private var nextId = 0

    fun createInitialReading(): RadiationReading {
        return sensor.getReading(nextId++)
    }

    fun generateNextReading(): RadiationReading {
        return sensor.getReading(nextId++)
    }
}
```

Тека com/example/npp_radiation_monitor/ui/viewmodel

Файл RadiationViewModel.kt

```
package com.example.npp_radiation_monitor.ui.viewmodel

import androidx.compose.runtime.getValue
import androidx.compose.runtime.mutableStateOf
import androidx.compose.runtime.setValue
import androidx.lifecycle.ViewModel
import com.example.npp_radiation_monitor.data.RadiationManager
import com.example.npp_radiation_monitor.data.model.RadiationReading

/**
 * ViewModel для управління станом показників радіаційного фону
(RadiationViewModel).
 * Зберігає поточний стан вимірювання радіації у пам'яті ViewModel,
 * що дозволяє уникати втрати даних при зміні конфігурації пристрою
(наприклад, при повороті екрана).
 */
class RadiationViewModel : ViewModel() {

    private val radiationManager = RadiationManager()

    // Поточний об'єкт вимірювання радіації (зберігається у ViewModel)
    var currentReading by
mutableStateOf(radiationManager.createInitialReading())
        private set

    /**
     * Генерує та оновлює нове значення радіаційного фону.
     */
    fun generateNextReading(): RadiationReading {
        val newReading = radiationManager.generateNextReading()
        currentReading = newReading
        return newReading
```

```
    }
}
```

Тека com/example/npp_radiation_monitor/ui/screens

Файл RadiationScreen.kt

```
package com.example.npp_radiation_monitor.ui.screens

import androidx.compose.foundation.layout.Column
import androidx.compose.foundation.layout.Row
import androidx.compose.foundation.layout.Spacer
import androidx.compose.foundation.layout.fillMaxSize
import androidx.compose.foundation.layout.fillMaxWidth
import androidx.compose.foundation.layout.height
import androidx.compose.foundation.layout.padding
import androidx.compose.foundation.layout.width
import androidx.compose.material.icons.Icons
import androidx.compose.material.icons.filled.AccountCircle
import androidx.compose.material.icons.filled.Warning
import androidx.compose.material3.Button
import androidx.compose.material3.HorizontalDivider
import androidx.compose.material3.Icon
import androidx.compose.material3.MaterialTheme
import androidx.compose.material3.Text
import androidx.compose.runtime.Composable
import androidx.compose.runtime.snapshots.SnapshotStateList
import androidx.compose.ui.Alignment
import androidx.compose.ui.Modifier
import androidx.compose.ui.text.font.FontWeight
import androidx.compose.ui.unit.dp
import androidx.compose.ui.unit.sp
import androidx.lifecycle.viewmodel.compose.viewModel
import androidx.navigation.NavController
import com.example.npp_radiation_monitor.data.model.RadiationReading
import com.example.npp_radiation_monitor.navigation.Screen
import com.example.npp_radiation_monitor.ui.components.StatusCard
import com.example.npp_radiation_monitor.ui.viewmodel.RadiationViewModel

/**
 * Екран моніторингу радіації (RadiationScreen).
 * Відображає поточні показники радіаційного фону, дозволяє генерувати
 * нові вимірювання за допомогою [RadiationViewModel] та зберігає їх у
спільну історію.
 */
@Composable
fun RadiationScreen(
    navController: NavController,
    history: SnapshotStateList<RadiationReading>,
    operatorName: String = "Черговий оператор",
    stationName: String = "Моніторинг АЕС",
```

```
    viewModel: RadiationViewModel = viewModel()
) {
    // Читаємо показники прямо з ViewModel (тепер стан зберігається при
повороті екрана!)
    val currentReading = viewModel.currentReading

    Column(
        modifier = Modifier
            .fillMaxSize()
            .padding(16.dp)
            .padding(top = 40.dp),
        horizontalAlignment = Alignment.CenterHorizontally,
    ) {
        // Рядок із інформацією про оператора
        Row(
            modifier = Modifier
                .fillMaxWidth()
                .padding(bottom = 8.dp),
            verticalAlignment = Alignment.CenterVertically
        ) {
            Icon(Icons.Default.AccountCircle, contentDescription = null)
            Spacer(modifier = Modifier.width(8.dp))
            Text(
                text = "Оператор: $operatorName",
                style = MaterialTheme.typography.titleMedium,
                fontWeight = FontWeight.SemiBold
            )
        }

        // Кнопка повернення на головну
        Button(
            onClick =
{ navController.navigate(Screen.MainScreen.route) },
            modifier = Modifier.fillMaxWidth()
        ) {
            Text(text = "Перейти на головну", fontSize = 16.sp)
        }

        HorizontalDivider(modifier = Modifier.padding(vertical = 12.dp))

        // Заголовок станції (передане значення)
        Text(
            text = stationName,
            style = MaterialTheme.typography.headlineMedium,
            fontWeight = FontWeight.Bold,
            modifier = Modifier.padding(bottom = 12.dp)
        )

        // Картка поточного стану радіації (зі збереженого стану у
ViewModel)
        StatusCard(reading = currentReading)
```

```
        Spacer(modifier = Modifier.height(12.dp))

        // Кнопка оновлення показників датчика
        Button(
            onClick = {
                val newReading = viewModel.generateNextReading()
                history.add(0, newReading)
            },
            modifier = Modifier.fillMaxWidth(),
            shape = MaterialTheme.shapes.large
        ) {
            Icon(Icons.Default.Warning, contentDescription = null)
            Spacer(Modifier.width(8.dp))
            Text("Оновити показники датчика")
        }

        Spacer(modifier = Modifier.height(16.dp))

        // Кнопка переходу до екрана історії
        Button(
            onClick =
{ navController.navigate(Screen.HistoryScreen.route) },
            modifier = Modifier.fillMaxWidth()
        ) {
            Text(text = "Переглянути історію", fontSize = 16.sp)
        }
    }
}
```

Тека com/example/npp_radiation_monitor/navigation

Файл Navigation.kt

```
package com.example.npp_radiation_monitor.navigation

import androidx.compose.runtime.Composable
import androidx.compose.runtime.mutableStateListOf
import androidx.compose.runtime.remember
import androidx.navigation.NavType
import androidx.navigation.compose.NavHost
import androidx.navigation.compose.composable
import androidx.navigation.compose.rememberNavController
import androidx.navigation.navArgument
import com.example.npp_radiation_monitor.data.model.RadiationReading
import com.example.npp_radiation_monitor.ui.screens.HistoryScreen
import com.example.npp_radiation_monitor.ui.screens.MainScreen
import com.example.npp_radiation_monitor.ui.screens.RadiationScreen
import com.example.npp_radiation_monitor.ui.screens.StationFormScreen

/**
```

```
 * Головний граф навігації застосунку.
 * Управляє переходами між екранами: MainScreen, RadiationScreen,
HistoryScreen та StationFormScreen.
 * Зберігає та передає стан історії вимірювань між екранами.
 */
@Composable
fun Navigation() {
    val navController = rememberNavController()

    // Спільний стан історії вимірювань та менеджер даних
    val history = remember { mutableStateListOf<RadiationReading>() }

    NavHost(navController = navController, startDestination =
Screen.MainScreen.route) {

        composable(route = Screen.MainScreen.route) {
            MainScreen(navController = navController)
        }

        composable(route = Screen.FormScreen.route) {
            StationFormScreen(navController = navController)
        }

        composable(
            route = Screen.RadiationScreen.route +
"/{stationName}/{operatorName}",
            arguments = listOf(
                navArgument("stationName") {
                    type = NavType.StringType
                    defaultValue = "Моніторинг АЕС"
                },
                navArgument("operatorName") {
                    type = NavType.StringType
                    defaultValue = "Черговий оператор"
                }
            )
        ) { entry ->
            // Отримуємо і назву станції, і ім'я оператора з аргументів
маршруту
            val stationName = entry.arguments?.getString("stationName")
                .orEmpty()
                .ifBlank { "Моніторинг АЕС" }
            val operatorName = entry.arguments?.getString("operatorName")
                .orEmpty()
                .ifBlank { "Черговий оператор" }

            RadiationScreen(
                navController = navController,
                history = history,
                operatorName = operatorName,
                stationName = stationName
```

```
            )
        }

        composable(route = Screen.HistoryScreen.route) {
            HistoryScreen(navController = navController, history =
history)
        }
    }
}
```

ПРОГРАМНИЙ КОД ПРАКТИЧНОГО ЗАНЯТТЯ №8

Тека com/example/npp_radiation_monitor

Файл MainActivity.kt

```
package com.example.npp_radiation_monitor

import android.os.Bundle
import androidx.activity.ComponentActivity
import androidx.activity.compose.setContent
import androidx.activity.enableEdgeToEdge
import com.example.npp_radiation_monitor.navigation.Navigation
import
com.example.npp_radiation_monitor.ui.theme.NPP_Radiation_MonitorTheme

class MainActivity : ComponentActivity() {
    override fun onCreate(savedInstanceState: Bundle?) {
        super.onCreate(savedInstanceState)
        enableEdgeToEdge()
        setContent {
            NPP_Radiation_MonitorTheme {
                Navigation()
            }
        }
    }
}
```

Тека com/example/npp_radiation_monitor/data

Файл RadiationManager.kt

```
package com.example.npp_radiation_monitor.data

import com.example.npp_radiation_monitor.data.model.RadiationReading
import com.example.npp_radiation_monitor.data.sensor.RadiationSensor
import
com.example.npp_radiation_monitor.data.sensor.SimulatedRadiationSensor

class RadiationManager(
    private val sensor: RadiationSensor = SimulatedRadiationSensor()
) {
    private var nextId = 0

    fun createInitialReading(): RadiationReading {
        return sensor.getReading(nextId++)
    }

    fun generateNextReading(): RadiationReading {
        return sensor.getReading(nextId++)
```

```
    }
}
```

Тека com/example/npp_radiation_monitor/data/local

Файл AppDatabase.kt

```
package com.example.npp_radiation_monitor.data.local

import android.content.Context
import androidx.room.Database
import androidx.room.Room
import androidx.room.RoomDatabase

/**
 * Головний клас бази даних Room (AppDatabase).
 * Реалізує шаблон Singleton для створення єдиного екземпляра бази даних.
 */
@Database(entities = [RadiationEntity::class], version = 1, exportSchema
= false)
abstract class AppDatabase : RoomDatabase() {

    abstract fun radiationDao(): RadiationDao

    companion object {
        @Volatile
        private var INSTANCE: AppDatabase? = null

        /**
         * Отримує або створює екземпляр бази даних Room.
         */
        fun getDatabase(context: Context): AppDatabase {
            return INSTANCE ?: synchronized(this) {
                val instance = Room.databaseBuilder(
                    context.applicationContext,
                    AppDatabase::class.java,
                    "npp_radiation_database"
                ).build()
                INSTANCE = instance
                instance
            }
        }
    }
}
```

Тека com/example/npp_radiation_monitor/data/local

Файл RadiationDao.kt

```
package com.example.npp_radiation_monitor.data.local

import androidx.room.Dao
import androidx.room.Insert
import androidx.room.OnConflictStrategy
import androidx.room.Query
import kotlinx.coroutines.flow.Flow

/**
 * DAO (Data Access Object) інтерфейс для виконання SQL-запитів у базі
даних Room.
 */
@Dao
interface RadiationDao {

    /**
     * Зберігає нове вимірювання радіації у базі даних.
     */
    @Insert(onConflict = OnConflictStrategy.REPLACE)
    suspend fun insertReading(reading: RadiationEntity)

    /**
     * Отримує всі збережені вимірювання з базі даних у вигляді
реактивного Flow списку.
     */
    @Query("SELECT * FROM radiation_readings ORDER BY id DESC")
    fun getAllReadings(): Flow<List<RadiationEntity>>
}
```

Тека com/example/npp_radiation_monitor/data/local

Файл RadiationEntity.kt

```
package com.example.npp_radiation_monitor.data.local

import androidx.room.Entity
import androidx.room.PrimaryKey

/**
 * Сутність Room Database (Entity), що описує таблицю збереження замірів
радіації.
 */
@Entity(tableName = "radiation_readings")
data class RadiationEntity(
    @PrimaryKey(autoGenerate = true)
    val id: Int = 0,
    val value: Float,
```

```
    val timestamp: String,
    val statusTitle: String
)
```

Тека com/example/npp_radiation_monitor/data/model

Файл RadiationReading.kt

```
package com.example.npp_radiation_monitor.data.model

data class RadiationReading(
    val id: Int,
    val value: Float,
    val timestamp: String,
    val status: RadiationStatus = RadiationStatus.fromValue(value)
)
```

Тека com/example/npp_radiation_monitor/data/model

Файл RadiationStatus.kt

```
package com.example.npp_radiation_monitor.data.model

import androidx.compose.ui.graphics.Color

enum class RadiationStatus(
    val title: String,
    val color: Color,
    val maxThreshold: Float
) {
    SAFE("Безпечно", Color(0xFF4CAF50), 0.30f),
    WARNING("Увага", Color(0xFFFFC107), 0.60f),
    DANGER("Небезпека", Color(0xFFF44336), Float.MAX_VALUE);

    companion object {
        fun fromValue(value: Float): RadiationStatus {
            return when {
                value < SAFE.maxThreshold -> SAFE
                value < WARNING.maxThreshold -> WARNING
                else -> DANGER
            }
        }
    }
}
```

Тека com/example/npp_radiation_monitor/data/repository

Файл RadiationRepository.kt

```
package com.example.npp_radiation_monitor.data.repository

import com.example.npp_radiation_monitor.data.local.RadiationDao
import com.example.npp_radiation_monitor.data.local.RadiationEntity
import kotlinx.coroutines.flow.Flow

/**
 * Репозиторій для роботи з даними радіаційного фону
(RadiationRepository).
 * Виступає посередником між джерелом даних (Room DAO) та ViewModel.
 */
class RadiationRepository(private val radiationDao: RadiationDao) {

    /**
     * Потік усіх збережених у базі вимірювань.
     */
    val allReadings: Flow<List<RadiationEntity>> =
radiationDao.getAllReadings()

    /**
     * Додає нове вимірювання у базу даних Room.
     */
    suspend fun insertReading(reading: RadiationEntity) {
        radiationDao.insertReading(reading)
    }
}
```

Тека com/example/npp_radiation_monitor/navigation

Файл Navigation.kt

```
package com.example.npp_radiation_monitor.navigation

import androidx.compose.runtime.Composable
import androidx.navigation.NavType
import androidx.navigation.compose.NavHost
import androidx.navigation.compose.composable
import androidx.navigation.compose.rememberNavController
import androidx.navigation.navArgument
import com.example.npp_radiation_monitor.ui.screens.HistoryScreen
import com.example.npp_radiation_monitor.ui.screens.MainScreen
import com.example.npp_radiation_monitor.ui.screens.RadiationScreen
import com.example.npp_radiation_monitor.ui.screens.StationFormScreen

/**
 * Головний граф навігації застосунку.
 * Управляє переходами між екранами: MainScreen, RadiationScreen,
```

```
HistoryScreen та StationFormScreen.
 */
@Composable
fun Navigation() {
    val navController = rememberNavController()

    NavHost(navController = navController, startDestination =
Screen.MainScreen.route) {

        composable(route = Screen.MainScreen.route) {
            MainScreen(navController = navController)
        }

        composable(route = Screen.FormScreen.route) {
            StationFormScreen(navController = navController)
        }

        composable(
            route = Screen.RadiationScreen.route +
"/{stationName}/{operatorName}",
            arguments = listOf(
                navArgument("stationName") {
                    type = NavType.StringType
                    defaultValue = "Моніторинг АЕС"
                },
                navArgument("operatorName") {
                    type = NavType.StringType
                    defaultValue = "Черговий оператор"
                }
            )
        ) { entry ->
            val stationName = entry.arguments?.getString("stationName")
                .orEmpty()
                .ifBlank { "Моніторинг АЕС" }
            val operatorName = entry.arguments?.getString("operatorName")
                .orEmpty()
                .ifBlank { "Черговий оператор" }

            RadiationScreen(
                navController = navController,
                operatorName = operatorName,
                stationName = stationName
            )
        }

        composable(route = Screen.HistoryScreen.route) {
            HistoryScreen(navController = navController)
        }
    }
}
```

Тека com/example/npp_radiation_monitor/navigation

Файл Screen.kt

```
package com.example.npp_radiation_monitor.navigation

sealed class Screen(val route: String) {
    object MainScreen : Screen("main_screen")
    object HistoryScreen : Screen("history_screen")
    object FormScreen : Screen("form_screen")
    object RadiationScreen : Screen("radiation_screen") {
        fun createRoute(
            stationName: String = "Моніторинг АЕС",
            operatorName: String = "Черговий оператор"
        ): String {
            val safeStation = stationName.ifBlank { "Моніторинг АЕС" }
            val safeOperator = operatorName.ifBlank { "Черговий оператор"
}
            return "radiation_screen/$safeStation/$safeOperator"
        }
    }
}
```

Тека com/example/npp_radiation_monitor/ui/components

Файл HistoryItem.kt

```
package com.example.npp_radiation_monitor.ui.components

import androidx.compose.foundation.layout.Arrangement
import androidx.compose.foundation.layout.Column
import androidx.compose.foundation.layout.Row
import androidx.compose.foundation.layout.fillMaxWidth
import androidx.compose.foundation.layout.padding
import androidx.compose.material3.Card
import androidx.compose.material3.CardDefaults
import androidx.compose.material3.MaterialTheme
import androidx.compose.material3.Text
import androidx.compose.runtime.Composable
import androidx.compose.ui.Alignment
import androidx.compose.ui.Modifier
import androidx.compose.ui.text.font.FontWeight
import androidx.compose.ui.unit.dp
import com.example.npp_radiation_monitor.data.model.RadiationReading
import java.util.Locale

/**
 * Окремий елемент списку історії вимірювань (HistoryItem).
 * Відображає номер вимірювання, час проведення та показник радіації з
кольором статусу.
```

```
 */
@Composable
fun HistoryItem(reading: RadiationReading) {
    Card(
        modifier = Modifier.fillMaxWidth(),
        colors = CardDefaults.cardColors(containerColor =
MaterialTheme.colorScheme.surfaceVariant)
    ) {
        Row(
            modifier = Modifier
                .padding(16.dp)
                .fillMaxWidth(),
            horizontalArrangement = Arrangement.SpaceBetween,
            verticalAlignment = Alignment.CenterVertically
        ) {
            Column {
                Text(text = "Замір #${reading.id}", style =
MaterialTheme.typography.labelMedium)
                Text(text = reading.timestamp, style =
MaterialTheme.typography.bodySmall)
            }
            Text(
                text = String.format(locale = Locale.UK, "%.2f мкЗв /
год", reading.value),
                style = MaterialTheme.typography.titleMedium,
                fontWeight = FontWeight.Bold,
                color = reading.status.color
            )
        }
    }
}
```

Тека com/example/npp_radiation_monitor/ui/components

Файл StatusCard.kt

```
package com.example.npp_radiation_monitor.ui.components

import androidx.compose.animation.animateColorAsState
import androidx.compose.animation.core.animateFloatAsState
import androidx.compose.foundation.layout.Arrangement
import androidx.compose.foundation.layout.Column
import androidx.compose.foundation.layout.fillMaxSize
import androidx.compose.foundation.layout.fillMaxWidth
import androidx.compose.foundation.layout.height
import androidx.compose.material3.Card
import androidx.compose.material3.CardDefaults
import androidx.compose.material3.MaterialTheme
import androidx.compose.material3.Text
import androidx.compose.runtime.Composable
import androidx.compose.runtime.getValue
```

```
import androidx.compose.ui.Alignment
import androidx.compose.ui.Modifier
import androidx.compose.ui.graphics.Color
import androidx.compose.ui.text.font.FontWeight
import androidx.compose.ui.unit.dp
import androidx.compose.ui.unit.sp
import com.example.npp_radiation_monitor.data.model.RadiationReading
import java.util.Locale

/**
 * Компонент картки статусу радіаційного фону (StatusCard).
 * Відображає поточний рівень радіації із анімованим зміненням кольору
 * та значення залежно від статусу безпеки.
 */
@Composable
fun StatusCard(
    reading: RadiationReading,
    modifier: Modifier = Modifier
) {
    // Анімації значення радіації та кольору картки
    val animatedRadiation by animateFloatAsState(targetValue =
reading.value)
    val animatedColor by animateColorAsState(targetValue =
reading.status.color)

    Card(
        modifier = modifier
            .fillMaxWidth()
            .height(180.dp),
        colors = CardDefaults.cardColors(containerColor = animatedColor),
        elevation = CardDefaults.cardElevation(defaultElevation = 8.dp)
    ) {
        Column(
            modifier = Modifier.fillMaxSize(),
            verticalArrangement = Arrangement.Center,
            horizontalAlignment = Alignment.CenterHorizontally
        ) {
            // Текст статусу (Норма, Увага, Небезпека)
            Text(
                text = "Статус: ${reading.status.title}",
                color = Color.White,
                style = MaterialTheme.typography.titleMedium
            )
            // Анімоване числове значення радіації
            Text(
                text = String.format(locale = Locale.UK, "%.2f",
animatedRadiation),
                color = Color.White,
                fontSize = 56.sp,
                fontWeight = FontWeight.Black
            )
```

```
            // Одиниці вимірювання
            Text(
                text = "мкЗв/год",
                color = Color.White,
                style = MaterialTheme.typography.bodyLarge
            )
        }
    }
}
```

Тека com/example/npp_radiation_monitor/ui/screens

Файл HistoryScreen.kt

```
package com.example.npp_radiation_monitor.ui.screens

import androidx.compose.foundation.layout.Arrangement
import androidx.compose.foundation.layout.Column
import androidx.compose.foundation.layout.Row
import androidx.compose.foundation.layout.Spacer
import androidx.compose.foundation.layout.fillMaxWidth
import androidx.compose.foundation.layout.padding
import androidx.compose.foundation.lazy.LazyColumn
import androidx.compose.foundation.lazy.items
import androidx.compose.material3.Button
import androidx.compose.material3.HorizontalDivider
import androidx.compose.material3.MaterialTheme
import androidx.compose.material3.Text
import androidx.compose.runtime.Composable
import androidx.compose.runtime.collectAsState
import androidx.compose.runtime.getValue
import androidx.compose.ui.Alignment
import androidx.compose.ui.Modifier
import androidx.compose.ui.text.font.FontWeight
import androidx.compose.ui.unit.dp
import androidx.compose.ui.unit.sp
import androidx.lifecycle.viewmodel.compose.viewModel
import androidx.navigation.NavController
import com.example.npp_radiation_monitor.data.model.RadiationReading
import com.example.npp_radiation_monitor.data.model.RadiationStatus
import com.example.npp_radiation_monitor.navigation.Screen
import com.example.npp_radiation_monitor.ui.components.HistoryItem
import com.example.npp_radiation_monitor.ui.viewmodel.RadiationViewModel

/**
 * Екран історії вимірювань (HistoryScreen).
 * Відображає список усіх збережених вимірювань радіаційного фону з
локальної бази даних Room.
 */
@Composable
fun HistoryScreen(
```

```
    navController: NavController,
    viewModel: RadiationViewModel = viewModel()
) {
    // Отримуємо реактивний список записів із локальної бази даних Room
    val roomReadings by
viewModel.allReadingsFromRoom.collectAsState(initial = emptyList())

    val history = roomReadings.map { entity ->
        RadiationReading(
            id = entity.id,
            value = entity.value,
            timestamp = entity.timestamp,
            status = RadiationStatus.fromValue(entity.value)
        )
    }

    Column(
        modifier = Modifier
            .fillMaxWidth()
            .padding(16.dp)
            .padding(top = 40.dp),
        horizontalAlignment = Alignment.CenterHorizontally
    ) {
        // Кнопка повернення на головний екран
        Button(
            onClick =
{ navController.navigate(Screen.MainScreen.route) },
            modifier = Modifier.fillMaxWidth()
        ) {
            Text(text = "Повернутися на головну", fontSize = 16.sp)
        }

        Spacer(modifier = Modifier.padding(8.dp))

        // Кнопка повернення на екран моніторингу
        Button(
            onClick = { navController.popBackStack() },
            modifier = Modifier.fillMaxWidth()
        ) {
            Text(text = "Повернутися до моніторингу радіації", fontSize =
16.sp)
        }

        Spacer(modifier = Modifier.padding(8.dp))

        // Заголовок екрана історії
        Row(
            modifier = Modifier.fillMaxWidth(),
            horizontalArrangement = Arrangement.Start
        ) {
            Text(
```

```
                text = "Історія вимірювань",
                style = MaterialTheme.typography.titleLarge,
                fontWeight = FontWeight.SemiBold
            )
        }

        HorizontalDivider(modifier = Modifier.padding(vertical = 8.dp))

        // Список усіх вимірювань із Room БД
        LazyColumn(
            modifier = Modifier
                .fillMaxWidth()
                .padding(bottom = 16.dp),
            verticalArrangement = Arrangement.spacedBy(8.dp)
        ) {
            items(history, key = { it.id }) { item ->
                HistoryItem(reading = item)
            }
        }
    }
}
```

Тека com/example/npp_radiation_monitor/ui/screens

Файл RadiationScreen.kt

```
package com.example.npp_radiation_monitor.ui.screens

import androidx.compose.foundation.layout.Column
import androidx.compose.foundation.layout.Row
import androidx.compose.foundation.layout.Spacer
import androidx.compose.foundation.layout.fillMaxSize
import androidx.compose.foundation.layout.fillMaxWidth
import androidx.compose.foundation.layout.height
import androidx.compose.foundation.layout.padding
import androidx.compose.foundation.layout.width
import androidx.compose.material.icons.Icons
import androidx.compose.material.icons.filled.AccountCircle
import androidx.compose.material.icons.filled.Warning
import androidx.compose.material3.Button
import androidx.compose.material3.HorizontalDivider
import androidx.compose.material3.Icon
import androidx.compose.material3.MaterialTheme
import androidx.compose.material3.Text
import androidx.compose.runtime.Composable
import androidx.compose.ui.Alignment
import androidx.compose.ui.Modifier
import androidx.compose.ui.text.font.FontWeight
import androidx.compose.ui.unit.dp
import androidx.compose.ui.unit.sp
import androidx.lifecycle.viewmodel.compose.viewModel
```

```
import androidx.navigation.NavController
import com.example.npp_radiation_monitor.navigation.Screen
import com.example.npp_radiation_monitor.ui.components.StatusCard
import com.example.npp_radiation_monitor.ui.viewmodel.RadiationViewModel

/**
 * Екран моніторингу радіації (RadiationScreen).
 * Відображає поточні показники радіаційного фону, дозволяє генерувати
 * нові вимірювання за допомогою RadiationViewModel та зберігає їх у базу
даних Room.
 */
@Composable
fun RadiationScreen(
    navController: NavController,
    operatorName: String = "Черговий оператор",
    stationName: String = "Моніторинг АЕС",
    viewModel: RadiationViewModel = viewModel()
) {
    // Читаємо показники прямо з ViewModel (стан зберігається у ViewModel
та Room)
    val currentReading = viewModel.currentReading

    Column(
        modifier = Modifier
            .fillMaxSize()
            .padding(16.dp)
            .padding(top = 40.dp),
        horizontalAlignment = Alignment.CenterHorizontally
    ) {
        // Рядок із інформацією про оператора
        Row(
            modifier = Modifier
                .fillMaxWidth()
                .padding(bottom = 8.dp),
            verticalAlignment = Alignment.CenterVertically
        ) {
            Icon(Icons.Default.AccountCircle, contentDescription = null)
            Spacer(modifier = Modifier.width(8.dp))
            Text(
                text = "Оператор: $operatorName",
                style = MaterialTheme.typography.titleMedium,
                fontWeight = FontWeight.SemiBold
            )
        }

        // Кнопка повернення на головну
        Button(
            onClick =
{ navController.navigate(Screen.MainScreen.route) },
            modifier = Modifier.fillMaxWidth()
        ) {
```

```
            Text(text = "Перейти на головну", fontSize = 16.sp)
        }

        HorizontalDivider(modifier = Modifier.padding(vertical = 12.dp))

        // Заголовок станції (передане значення)
        Text(
            text = stationName,
            style = MaterialTheme.typography.headlineMedium,
            fontWeight = FontWeight.Bold,
            modifier = Modifier.padding(bottom = 12.dp)
        )

        // Картка поточного стану радіації
        StatusCard(reading = currentReading)

        Spacer(modifier = Modifier.height(12.dp))

        // Кнопка оновлення показників датчика
        Button(
            onClick = {
                viewModel.generateNextReading()
            },
            modifier = Modifier.fillMaxWidth(),
            shape = MaterialTheme.shapes.large
        ) {
            Icon(Icons.Default.Warning, contentDescription = null)
            Spacer(Modifier.width(8.dp))
            Text("Оновити показники датчика")
        }

        Spacer(modifier = Modifier.height(16.dp))

        // Кнопка переходу до екрана історії
        Button(
            onClick =
{ navController.navigate(Screen.HistoryScreen.route) },
            modifier = Modifier.fillMaxWidth()
        ) {
            Text(text = "Переглянути історію", fontSize = 16.sp)
        }
    }
}
```

Тека com/example/npp_radiation_monitor/ui/viewmodel

Файл RadiationViewModel.kt

```
package com.example.npp_radiation_monitor.ui.viewmodel

import android.app.Application
```

```
import androidx.compose.runtime.getValue
import androidx.compose.runtime.mutableStateOf
import androidx.compose.runtime.setValue
import androidx.lifecycle.AndroidViewModel
import androidx.lifecycle.viewModelScope
import com.example.npp_radiation_monitor.data.RadiationManager
import com.example.npp_radiation_monitor.data.local.AppDatabase
import com.example.npp_radiation_monitor.data.local.RadiationEntity
import com.example.npp_radiation_monitor.data.model.RadiationReading
import
com.example.npp_radiation_monitor.data.repository.RadiationRepository
import kotlinx.coroutines.flow.Flow
import kotlinx.coroutines.launch

/**
 * ViewModel для управління станом показників радіаційного фону та
локальною базою даних Room (RadiationViewModel).
 * Інтегровано з RadiationRepository для збереження замірів у локальне
сховище Room Database.
 */
class RadiationViewModel(application: Application) :
AndroidViewModel(application) {

    private val radiationManager = RadiationManager()
    private val repository: RadiationRepository

    // Поточний об'єкт вимірювання радіації
    var currentReading by
mutableStateOf(radiationManager.createInitialReading())
        private set

    // Реактивний потік усіх записів із бази даних Room
    val allReadingsFromRoom: Flow<List<RadiationEntity>>

    init {
        val dao = AppDatabase.getDatabase(application).radiationDao()
        repository = RadiationRepository(dao)
        allReadingsFromRoom = repository.allReadings
    }

    /**
     * Генерує нове значення радіаційного фону та зберігає його у
локальну базу даних Room.
     */
    fun generateNextReading(): RadiationReading {
        val newReading = radiationManager.generateNextReading()
        currentReading = newReading

        // Зберігаємо замір у локальну базу Room (асинхронно через
Coroutine)
        viewModelScope.launch {
```

```
            repository.insertReading(
                RadiationEntity(
                    value = newReading.value,
                    timestamp = newReading.timestamp,
                    statusTitle = newReading.status.title
                )
            )
        }

        return newReading
    }
}
```